

lista_2_clean

June 7, 2025

```
[ ]: colab = True

import os

if colab:
    !git clone https://github.com/mdrs-thiago/PUC_Redes_Neuraais
    os.chdir('/content/PUC_Redes_Neuraais/pos_grad/lista 2')
```

1 PUC-Rio

1.1 Departamento de Engenharia Elétrica

1.2 Trabalho 2 - Previsão de séries temporais

Estudante:

Base de dados:

```
[81]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.utils.data import WeightedRandomSampler, Dataset, DataLoader
import torch.nn.functional as F

from sklearn.metrics import mean_squared_error, mean_absolute_error
import os

from model_utils import train, train_multi_step_model

from tqdm.notebook import tqdm
```

1.3 Preparação para a lista

```
[82]: class CustomDataset(Dataset):
```

```
    def __init__(self, X_data, y_data):
        self.X_data = X_data
        self.y_data = y_data

    def __getitem__(self, index):
        return self.X_data[index], self.y_data[index]

    def __len__(self):
        return len(self.X_data)
```

```
[83]: class NeuralNetwork(torch.nn.Module):
```

```
    '''
    Objeto criado para facilitar o desenvolvimento dos scripts das aulas práticas.
    Opção alternativa à criação externa fdo modelo. Basicamente serve como um
    'agregador' de trechos comuns para a criação do modelo. Por exemplo, ao invés
    de gastar n+1 linhas para a criação de um modelo com n camadas, podemos criar
    o mesmo modelo com apenas uma linha.

    Parâmetros de entrada:
    - hidden_neurons: Lista com a quantidade de neurônios na camada escondida.
    - hidden_activation: Função de ativação para cada camada escondida. Aceita
    como parâmetro string ou lista. Caso o parâmetro receba string, então a mesma
    função de ativação é utilizada para todas as camadas escondidas. Caso seja
    ↪ uma
    lista, cada camada terá sua própria função de ativação definida pela lista.
    - output_activation: Função de ativação para a camada de saída.
    - lr: Taxa de aprendizado
    - n_input: Tamanho do vetor de entrada.
    - n_output: Saída do modelo.
    '''

    def __init__(self, hidden_neurons = 4, hidden_activation = 'relu',
    ↪ output_activation='softmax', lr = 0.05, n_input = 1, n_output = 1):
        # create model
        super(NeuralNetwork, self).__init__()

        self.activations = {'relu': nn.ReLU(), 'sigmoid':nn.Sigmoid(), 'softmax':nn.
        ↪ Softmax()}

        self.fc1 = nn.Linear(n_input, hidden_neurons)
        self.ha = self.activations[hidden_activation]
        self.fc2 = nn.Linear(hidden_neurons, n_output)
        #self.out = self.activations[output_activation]
```

```

        #self.out = nn.Sigmoid() #Para o caso binário
def forward(self,x):
    h = self.fc1(x)
    h1 = self.ha(h)
    y = self.fc2(h1)
    #y = self.out(h2)
    return y

```

```

[84]: def transform_data(data, lag = 1, test_split = 0.1, normalize = True, category_
      => 'binary'):

```

```

    """
    Função para transformação de dados para a 2ª Lista de exercícios.
    A transformação de dados nesta etapa tem duas funções principais: a_
    ↳ transformação da série em uma forma tabular para
      o aprendizado de Redes Neurais e normalização dos dados.
    Parâmetros de entrada:
    - data: base de dados a ser ajeitado.
    - test_split: propoporção da base de dados para teste. Caso seja um float_
    ↳ entre 0 e 1, o valor é interpretado como proporção.
      Caso seja um valor inteiro, é compreendido como o tempo reservado para_
    ↳ generalização (e.g. lag = 12 reserva os 12 últimos
      registros para o teste).
    - normalize: normalização da série. Default True
    - categ: string contendo a codificação a ser realizada para a variável_
    ↳ exógena (no caso, o mês).

    A função retorna dois dataframes, sendo o primeiro para treinamento e o_
    ↳ segundo para generalização.
    """

    global _min
    global _max

    data_values = data.values

    if 0 < test_split < 1.0:
        l = data_values.shape[0]
        train_values = data_values[:-int(test_split*l),0].reshape(-1,1)
        test_values = data_values[-int(test_split*l):,0].reshape(-1,1)
        train_idx = data_values[:-int(test_split*l),1].reshape(-1,1)
        test_idx = data_values[-int(test_split*l):,1].reshape(-1,1)

    elif test_split > 1 and type(test_split) is int:

```

```

train_values = data_values[:-test_split,0].reshape(-1,1)
test_values = data_values[-test_split:,0].reshape(-1,1)
train_idx = data_values[:-test_split,1].reshape(-1,1)
test_idx = data_values[-test_split:,1].reshape(-1,1)

else:
    print('Test split not understood. Test split should be float between 0_
↪and 1 or integer for index')

assert test_values.shape[0] >= (lag)

_min = np.min(train_values)
_max = np.max(train_values)

if normalize:

    test_values = (test_values - _min)/(_max - _min)
    train_values = (train_values - _min)/(_max - _min)

train_data = np.zeros((train_values.shape[0] - (lag + 1), lag + 2))
test_data = np.zeros((test_values.shape[0], lag + 2))

all_data = np.vstack((train_values,test_values))
all_idx = np.vstack((train_idx,test_idx))

new_data = np.zeros((train_values.shape[0] - (lag + 1) + test_values.
↪shape[0],lag + 2))

for i in range(lag + 2):
    new_data[:,i] = all_data[i:new_data.shape[0]+i,0]

if category == 'binary':
    binary_rep = [np.binary_repr(z,width=4) for z in all_idx.astype('int').
↪reshape(-1,)]
    t0 = np.array([int(v) for s in binary_rep for v in s[0]])
    t1 = np.array([int(v) for s in binary_rep for v in s[1]])
    t2 = np.array([int(v) for s in binary_rep for v in s[2]])
    t3 = np.array([int(v) for s in binary_rep for v in s[3]])
    t = np.vstack((t0,t1,t2,t3)).T
    t = t[-new_data.shape[0]:,:]
    temp_idx = [f'month_{i}' for i in range(4)]

```

```

elif category == '1toN':
    pass

else:
    t = (all_idx - np.min(train_idx))/(np.max(train_idx) - np.
    ↪min(train_idx))
    t = t.reshape(-1,1)
    t = t[-new_data.shape[0]:,:]
    temp_idx = ['month']

    new_data = np.hstack((t,new_data))

    train_data = new_data[:-test_values.shape[0],:]
    test_data = new_data[-test_values.shape[0]:,:]

    data_columns = [f'y(t{+i})' if i < 0 else 'y(t)' if i == 0 else f'y(t+{i})' ]
    ↪for i in range(-lag,2)]
    temp_idx.extend(data_columns)
    new_train_df = pd.DataFrame(train_data, columns=temp_idx)
    new_test_df = pd.DataFrame(test_data, columns=temp_idx)

    return new_train_df, new_test_df

```

```

[85]: dataset_name = 'microclima1'
path = f'https://raw.githubusercontent.com/mdrs-thiago/PUC_Redes_Neuralis/main/
    ↪datasets/s_{dataset_name}.csv'
raw_data = pd.read_csv(path)

```

1.4 Parte I - Compreensão do problema

Total: 3,0 pts.

1. Visualize, em forma de gráfico, a dinâmica temporal da série escolhida. A série é adequada para a modelagem usando Redes Neurais? Caso não seja, que técnicas podem ser aplicadas para ajustar o comportamento da série?

```
[7]: raw_data
```

```

[7]:
   value  time
0  42.001613  1.0
1  42.300000  2.0
2  39.791935  3.0
3  35.808333  4.0
4  29.591935  5.0
..      ...  ...

```

```

139 29.095161 8.0
140 33.425000 9.0
141 32.254839 10.0
142 40.753333 11.0
143 37.708065 12.0

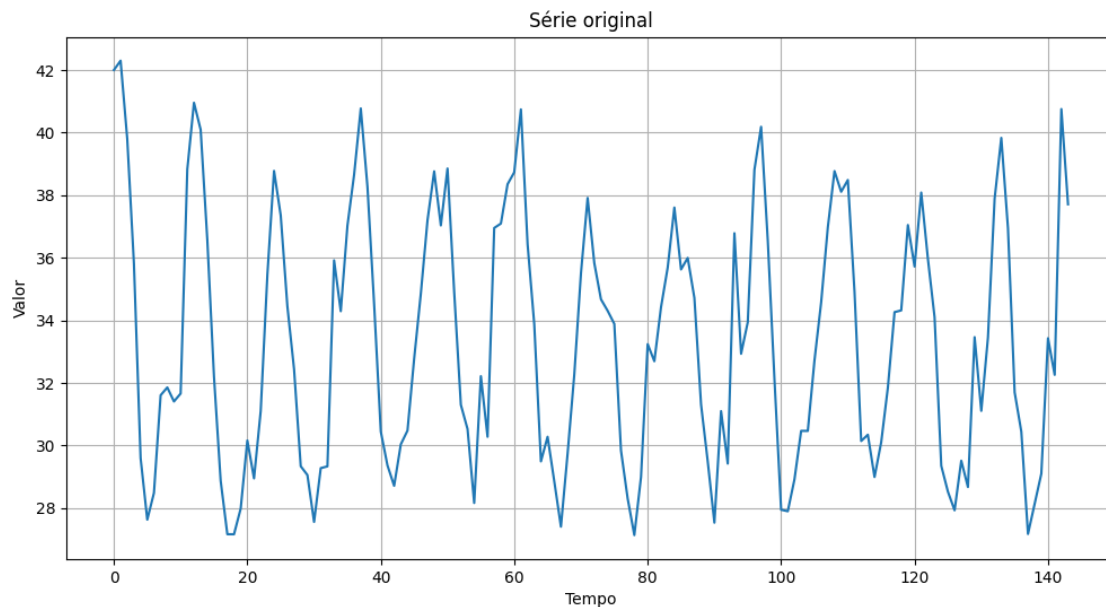
```

```
[144 rows x 2 columns]
```

```

[7]: plt.figure(figsize=(12,6))
plt.plot(raw_data['value'])
plt.title('Série original')
plt.xlabel('Tempo')
plt.ylabel('Valor')
plt.grid()
plt.show()

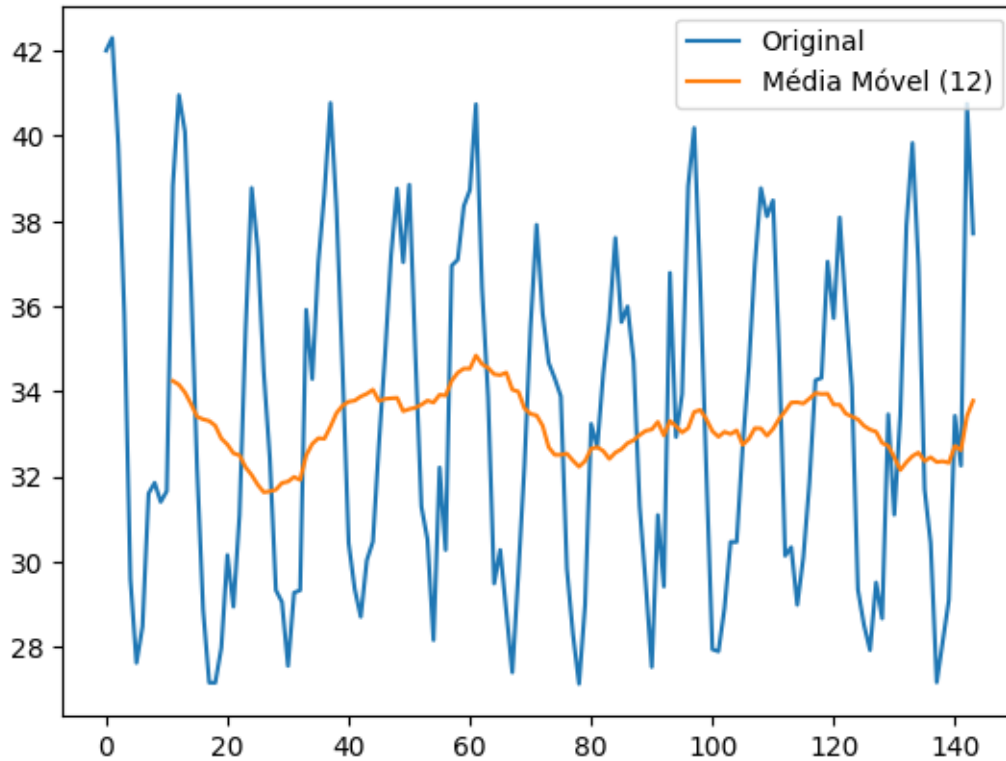
```



```

[8]: raw_data['moving_avg'] = raw_data['value'].rolling(window=12).mean()
plt.plot(raw_data['value'], label='Original')
plt.plot(raw_data['moving_avg'], label='Média Móvel (12)')
plt.legend()
plt.show()

```



```
[9]: from statsmodels.tsa.seasonal import seasonal_decompose
result = seasonal_decompose(raw_data['value'], period=12)
result.trend.plot(title='Tendência')
result.seasonal.plot(title='Componente Sazonal')
```

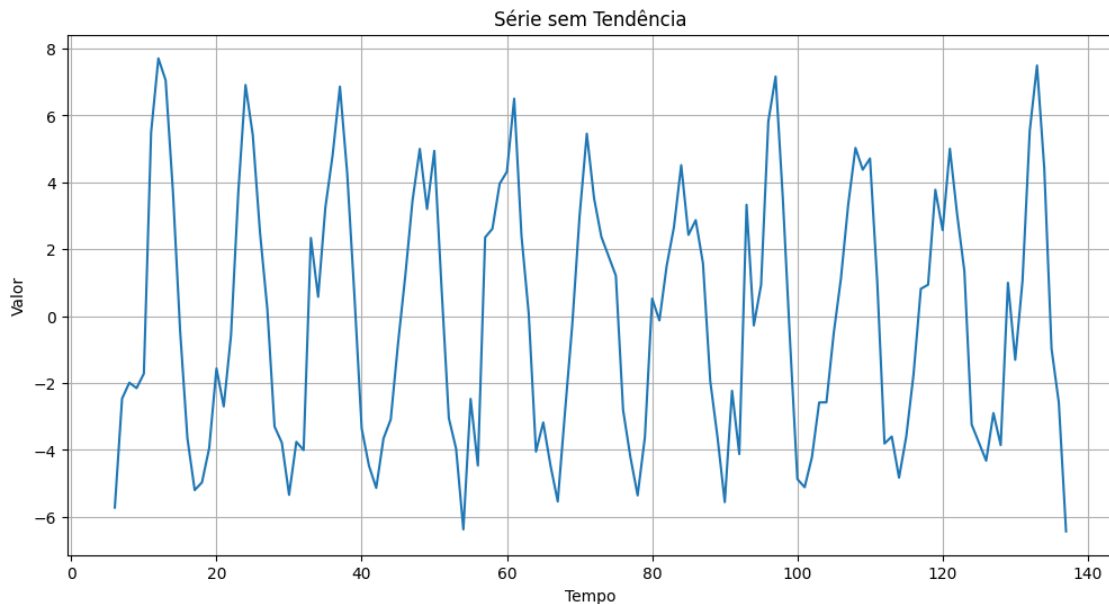
```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 from statsmodels.tsa.seasonal import seasonal_decompose
      2 result = seasonal_decompose(raw_data['value'], period=12)
      3 result.trend.plot(title='Tendência')

ModuleNotFoundError: No module named 'statsmodels'
```

```
[11]: sazonal_var = np.var(result.seasonal.dropna())
total_var = np.var(raw_data['value'])
print(f'Proporção da variância sazonal: {sazonal_var/total_var:.2f}')
```

Proporção da variância sazonal: 0.79

```
[12]: serie_sem_tendencia = raw_data['value'] - result.trend
plt.figure(figsize=(12,6))
plt.plot(serie_sem_tendencia, label='Série sem Tendência')
plt.title('Série sem Tendência')
plt.xlabel('Tempo')
plt.ylabel('Valor')
plt.grid()
plt.show()
```



```
[13]: from scipy.stats import linregress

trend = result.trend.dropna()
slope, intercept, r_value, p_value, std_err = linregress(range(len(trend)),
↪trend)
print(f"Inclinação: {slope:.4f}, R²: {r_value**2:.4f}")
```

Inclinação: -0.0014, R²: 0.0060

Não. Pois falta retirar a tendência da série temporal. A tendência em uma série temporal pode prejudicar o aprendizado pois impossibilita o modelo de aprender variações sutis nos dados, focando apenas em tendências crescentes ou decrescentes de longo prazo. Além disso, remover a tendência torna o processo estocástico mais estacionário, mantendo métricas estatísticas média e variância constantes e covariância invariantes a deslocamento.

2. Nesta 2ª lista de exercícios, usaremos uma variável exógena que representa o mês de previsão (i.e. no instante $t+1$). De que forma esta variável pode auxiliar na previsão da série temporal? Como nosso conjunto de dados consiste nas temperaturas ao longo do tempo, esperamos que ao usar uma variável exógena representando o mês ajudemos na

previsão da temperatura, pois sabemos que o mês atual está relacionado com as estações do ano e consequentemente à temperatura.

3. Observe que a base de dados não está em um formato adequado para o uso imediato de um modelo de Rede Neural para previsão de séries temporais. Descreva as etapas necessárias de transformação desta base de dados para que possamos utilizá-la no treinamento de uma MLP. Considere que os padrões de entrada (e saída) devem ser normalizados. É necessário definir as janelas de entrada e horizonte de previsão adequados e gerar várias amostras a partir dessas janelas deslizantes ao longo do ponto.

1.5 Parte 2 - Previsão one-step

Total: 2,0 pts

IMPORTANTE! Este processo não é a previsão multi-step, conforme ensinado em sala de aula. Esta parte da lista é importante para evitar erros de avaliação para a previsão de séries temporais com um horizonte de previsão maior que 1.

1. Execute o script para a previsão one-step. Avalie o resultado (conjunto de treinamento e teste) usando métricas apropriadas, como RMSE e MAE.

```
[14]: prev = 12
train_val ,test_val = transform_data(raw_data,lag=12,test_split =
    ↪prev,normalize=True,category='binary')
```

```
[15]: train_val
```

```
[15]:
```

	month_0	month_1	month_2	month_3	y(t-12)	y(t-11)	y(t-10)	\
0	0.0	0.0	1.0	0.0	0.980330	1.000000	0.834666	
1	0.0	0.0	1.0	1.0	1.000000	0.834666	0.572063	
2	0.0	1.0	0.0	0.0	0.834666	0.572063	0.162272	
3	0.0	1.0	0.0	1.0	0.572063	0.162272	0.032500	
4	0.0	1.0	1.0	0.0	0.162272	0.032500	0.088908	
..	...	...	...	...	...	...	...	
114	1.0	0.0	0.0	0.0	0.122400	0.195764	0.310928	
115	1.0	0.0	0.0	1.0	0.195764	0.310928	0.469974	
116	1.0	0.0	1.0	0.0	0.310928	0.469974	0.473621	
117	1.0	0.0	1.0	1.0	0.469974	0.473621	0.653596	
118	1.0	1.0	0.0	0.0	0.473621	0.653596	0.566091	

	y(t-9)	y(t-8)	y(t-7)	y(t-6)	y(t-5)	y(t-4)	y(t-3)	\
0	0.572063	0.162272	0.032500	0.088908	0.294752	0.311236	0.281674	
1	0.162272	0.032500	0.088908	0.294752	0.311236	0.281674	0.298381	
2	0.032500	0.088908	0.294752	0.311236	0.281674	0.298381	0.771084	
3	0.088908	0.294752	0.311236	0.281674	0.298381	0.771084	0.911432	
4	0.294752	0.311236	0.281674	0.298381	0.771084	0.911432	0.854621	
..	...	...	...	...	...	...	...	
114	0.469974	0.473621	0.653596	0.566091	0.721995	0.582359	0.459338	
115	0.473621	0.653596	0.566091	0.721995	0.582359	0.459338	0.146111	

```

116 0.653596 0.566091 0.721995 0.582359 0.459338 0.146111 0.091499
117 0.566091 0.721995 0.582359 0.459338 0.146111 0.091499 0.052226
118 0.721995 0.582359 0.459338 0.146111 0.091499 0.052226 0.156849

```

```

      y(t-2)  y(t-1)      y(t)  y(t+1)
0  0.298381 0.771084 0.911432 0.854621
1  0.771084 0.911432 0.854621 0.622018
2  0.911432 0.854621 0.622018 0.333649
3  0.854621 0.622018 0.333649 0.114426
4  0.622018 0.333649 0.114426 0.001846
..      ...      ...      ...      ...
114 0.146111 0.091499 0.052226 0.156849
115 0.091499 0.052226 0.156849 0.101277
116 0.052226 0.156849 0.101277 0.417237
117 0.156849 0.101277 0.417237 0.261905
118 0.101277 0.417237 0.261905 0.416387

```

[119 rows x 18 columns]

```
[16]: test_val
```

```

[16]:      month_0  month_1  month_2  month_3  y(t-12)  y(t-11)  y(t-10)  \
0         0.0        0.0        0.0        1.0  0.653596  0.566091  0.721995
1         0.0        0.0        1.0        0.0  0.566091  0.721995  0.582359
2         0.0        0.0        1.0        1.0  0.721995  0.582359  0.459338
3         0.0        1.0        0.0        0.0  0.582359  0.459338  0.146111
4         0.0        1.0        0.0        1.0  0.459338  0.146111  0.091499
5         0.0        1.0        1.0        0.0  0.146111  0.091499  0.052226
6         0.0        1.0        1.0        1.0  0.091499  0.052226  0.156849
7         1.0        0.0        0.0        0.0  0.052226  0.156849  0.101277
8         1.0        0.0        0.0        1.0  0.156849  0.101277  0.417237
9         1.0        0.0        1.0        0.0  0.101277  0.417237  0.261905
10        1.0        0.0        1.0        1.0  0.417237  0.261905  0.416387
11        1.0        1.0        0.0        0.0  0.261905  0.416387  0.709203

      y(t-9)  y(t-8)  y(t-7)  y(t-6)  y(t-5)  y(t-4)  y(t-3)  \
0  0.582359  0.459338  0.146111  0.091499  0.052226  0.156849  0.101277
1  0.459338  0.146111  0.091499  0.052226  0.156849  0.101277  0.417237
2  0.146111  0.091499  0.052226  0.156849  0.101277  0.417237  0.261905
3  0.091499  0.052226  0.156849  0.101277  0.417237  0.261905  0.416387
4  0.052226  0.156849  0.101277  0.417237  0.261905  0.416387  0.709203
5  0.156849  0.101277  0.417237  0.261905  0.416387  0.709203  0.837316
6  0.101277  0.417237  0.261905  0.416387  0.709203  0.837316  0.646897
7  0.417237  0.261905  0.416387  0.709203  0.837316  0.646897  0.301018
8  0.261905  0.416387  0.709203  0.837316  0.646897  0.301018  0.218092
9  0.416387  0.709203  0.837316  0.646897  0.301018  0.218092  0.002396
10 0.709203  0.837316  0.646897  0.301018  0.218092  0.002396  0.065517

```

```
11  0.837316  0.646897  0.301018  0.218092  0.002396  0.065517  0.129524
```

	y(t-2)	y(t-1)	y(t)	y(t+1)
0	0.417237	0.261905	0.416387	0.709203
1	0.261905	0.416387	0.709203	0.837316
2	0.416387	0.709203	0.837316	0.646897
3	0.709203	0.837316	0.646897	0.301018
4	0.837316	0.646897	0.301018	0.218092
5	0.646897	0.301018	0.218092	0.002396
6	0.301018	0.218092	0.002396	0.065517
7	0.218092	0.002396	0.065517	0.129524
8	0.002396	0.065517	0.129524	0.414951
9	0.065517	0.129524	0.414951	0.337813
10	0.129524	0.414951	0.337813	0.898042
11	0.414951	0.337813	0.898042	0.697295

```
[17]: train_values = train_val.values
```

```
X_train = train_values[:, :-1]
y_train = train_values[:, -1]
```

```
test_values = test_val.values
X_test = test_values[:, :-1]
y_test = test_values[:, -1]
```

```
[18]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
    ↪from_numpy(y_train).float().unsqueeze(1))
```

```
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
    ↪from_numpy(y_test).float().unsqueeze(1))
```

```
[19]: batch_size = 32
```

```
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
```

```
test_loader = DataLoader(dataset=test_dataset, batch_size=1)
```

```
[20]: n_input = X_train.shape[1]
```

```
n_output = 1
```

```
hidden_neurons = 20
```

```
[21]: model = NeuralNetwork(n_input = n_input, n_output=n_output,
    ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid')
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
model.to(device)
```

```
print(device)
```

```
lr = 0.05
```

```
epochs = 1000

history, y_hat = train(model, train_loader, epochs, device, lr)
```

cuda

```
 8%|          | 80/1000 [00:01<00:07, 116.48it/s]
Epoch 51/1000, Loss: 0.0292
12%|          | 120/1000 [00:01<00:05, 153.94it/s]
Epoch 101/1000, Loss: 0.0138
18%|          | 176/1000 [00:01<00:04, 171.09it/s]
Epoch 151/1000, Loss: 0.0093
21%|          | 212/1000 [00:01<00:04, 160.87it/s]
Epoch 201/1000, Loss: 0.0087
28%|          | 283/1000 [00:02<00:04, 170.07it/s]
Epoch 251/1000, Loss: 0.0085
32%|          | 321/1000 [00:02<00:03, 179.51it/s]
Epoch 301/1000, Loss: 0.0084
38%|          | 376/1000 [00:02<00:03, 171.85it/s]
Epoch 351/1000, Loss: 0.0083
43%|          | 430/1000 [00:03<00:03, 167.22it/s]
Epoch 401/1000, Loss: 0.0083
47%|          | 466/1000 [00:03<00:03, 167.83it/s]
Epoch 451/1000, Loss: 0.0082
52%|          | 519/1000 [00:03<00:02, 167.46it/s]
Epoch 501/1000, Loss: 0.0082
57%|          | 574/1000 [00:04<00:02, 164.52it/s]
Epoch 551/1000, Loss: 0.0081
63%|          | 626/1000 [00:04<00:02, 153.24it/s]
Epoch 601/1000, Loss: 0.0081
67%|          | 672/1000 [00:04<00:02, 128.58it/s]
Epoch 651/1000, Loss: 0.0081
73%|          | 726/1000 [00:05<00:01, 160.46it/s]
Epoch 701/1000, Loss: 0.0080
```

```

78%|      | 781/1000 [00:05<00:01, 161.86it/s]
Epoch 751/1000, Loss: 0.0080
82%|      | 815/1000 [00:05<00:01, 159.22it/s]
Epoch 801/1000, Loss: 0.0080
85%|      | 848/1000 [00:05<00:01, 144.45it/s]
Epoch 851/1000, Loss: 0.0080
94%|      | 938/1000 [00:06<00:00, 158.73it/s]
Epoch 901/1000, Loss: 0.0080
97%|      | 973/1000 [00:06<00:00, 162.76it/s]
Epoch 951/1000, Loss: 0.0079
100%|     | 1000/1000 [00:06<00:00, 143.15it/s]
Epoch 1001/1000, Loss: 0.0079

```

```

[22]: y_pred_tr = model(torch.from_numpy(X_train).float().to(device))
      y_hat_tr = y_pred_tr.detach().cpu().numpy()

      orig_y_eval = y_train*(_max-_min) + _min
      orig_y_hat = y_hat_tr*(_max-_min) + _min
      mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
      mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

      plt.plot(orig_y_eval)
      plt.plot(orig_y_hat)
      plt.legend(['Original', 'Predicted'])

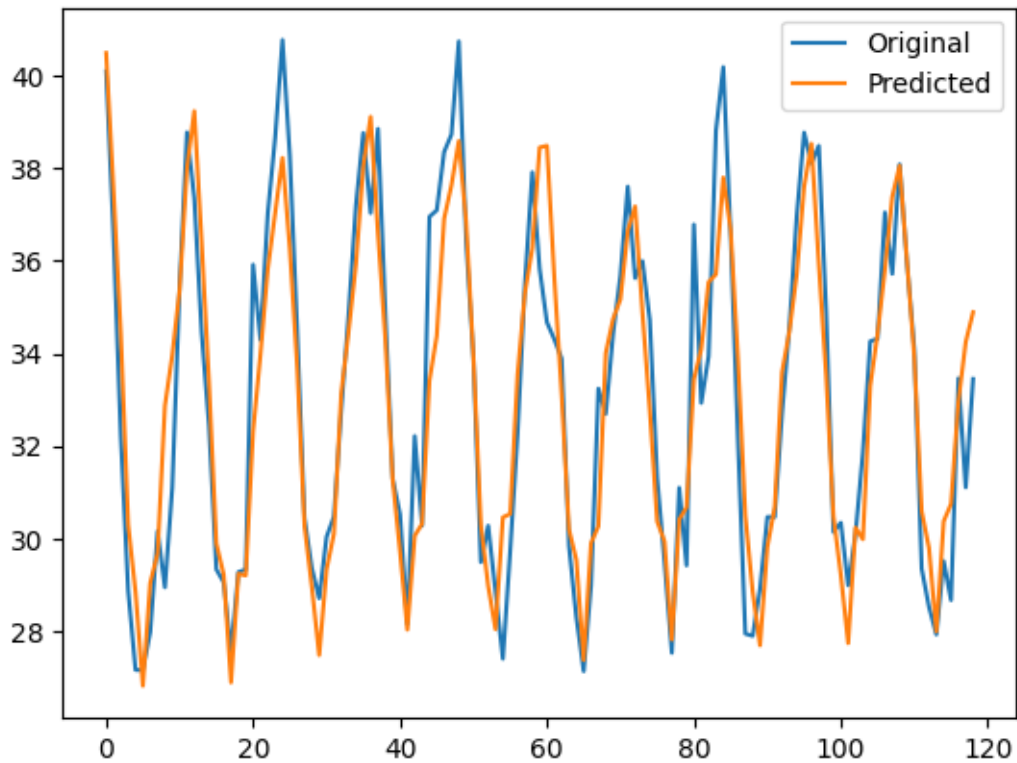
      print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

```

Erro MSE = 2.314
Erro MAE = 1.19

```



```
[23]: y_pred = model(torch.from_numpy(X_test).float().to(device))
      y_hat = y_pred.detach().cpu().numpy()

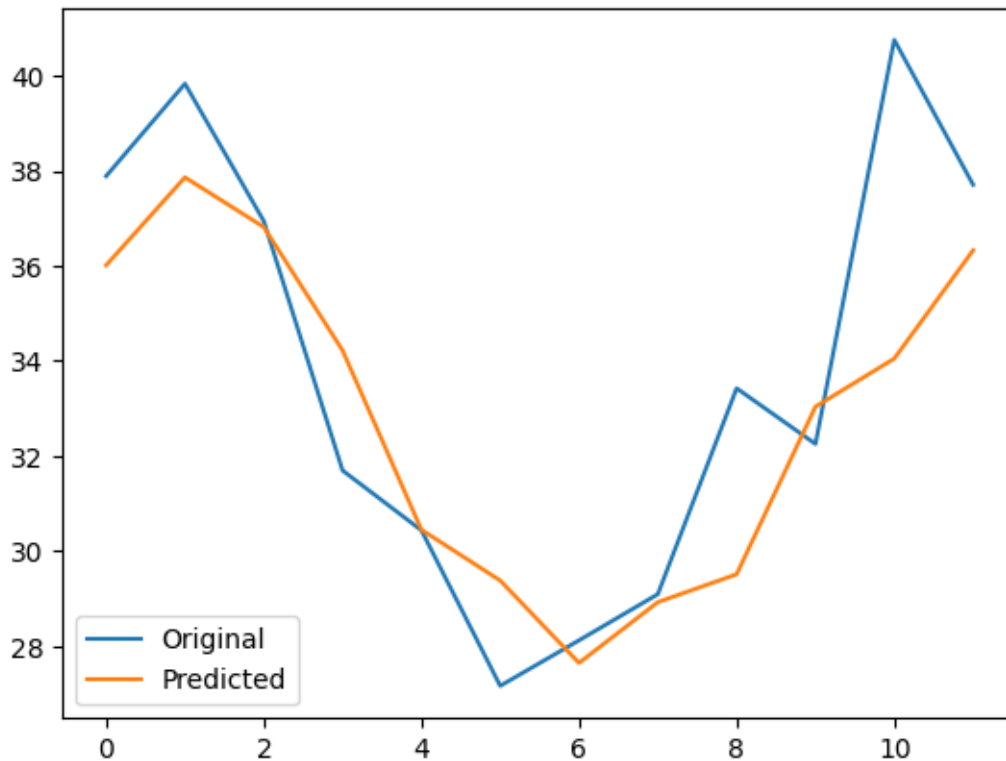
      orig_y_eval = y_test*(_max-_min) + _min
      orig_y_hat = y_hat*(_max-_min) + _min
      mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
      mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

      plt.plot(orig_y_eval)
      plt.plot(orig_y_hat)
      plt.legend(['Original', 'Predicted'])

      print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')
```

Erro MSE = 6.809

Erro MAE = 1.846



Podemos ver que o modelo aprendeu bem no conjunto de treino, entretanto no conjunto de teste performou um pouco pior. Apesar da falta de generalização o modelo mostrou aprender padrões básicos como tendências de decrescimento e crescimento da série em torno do tempo 5.

2. Modifique a técnica de codificação mensal de ‘binário’ para ‘numérico’. Qual a mudança existente na arquitetura da Rede Neural? Analise o resultado (conjunto de treinamento e teste), usando as métricas adequadas.

```
[24]: prev = 12
train_val ,test_val = transform_data(raw_data, lag=12, test_split = 0.2,
    ↪prev,normalize=True,category='numeric')
```

```
[25]: train_values = train_val.values

X_train = train_values[:, :-1]
y_train = train_values[:, -1]

test_values = test_val.values
X_test = test_values[:, :-1]
y_test = test_values[:, -1]
```

```
[26]: print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
```

```
(119, 14) (119,)
(12, 14) (12,)
```

```
[27]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))

test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))
```

```
[28]: batch_size = 32

train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)

test_loader = DataLoader(dataset=test_dataset, batch_size=1)
```

```
[29]: n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 20

model = NeuralNetwork(n_input = n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000

history, y_hat = train(model, train_loader, epochs, device, lr)
```

cuda

```
0%|          | 0/1000 [00:00<?, ?it/s]  7%|          | 74/1000 [00:00<00:06,
142.94it/s]
```

Epoch 51/1000, Loss: 0.0302

```
12%|          | 123/1000 [00:00<00:06, 145.27it/s]
```

Epoch 101/1000, Loss: 0.0163

```
17%|          | 167/1000 [00:01<00:06, 128.63it/s]
```

Epoch 151/1000, Loss: 0.0119

```
22%|          | 219/1000 [00:01<00:06, 125.17it/s]
```

Epoch 201/1000, Loss: 0.0115

```
28%|          | 275/1000 [00:02<00:05, 132.20it/s]
```

Epoch 251/1000, Loss: 0.0116

```
32%|          | 319/1000 [00:02<00:04, 139.74it/s]
```

Epoch 301/1000, Loss: 0.0116
 38%| | 381/1000 [00:02<00:04, 146.96it/s]
 Epoch 351/1000, Loss: 0.0115
 43%| | 427/1000 [00:03<00:04, 140.41it/s]
 Epoch 401/1000, Loss: 0.0115
 48%| | 480/1000 [00:03<00:03, 164.58it/s]
 Epoch 451/1000, Loss: 0.0114
 54%| | 536/1000 [00:03<00:02, 177.84it/s]
 Epoch 501/1000, Loss: 0.0113
 57%| | 572/1000 [00:03<00:02, 172.46it/s]
 Epoch 551/1000, Loss: 0.0113
 63%| | 630/1000 [00:04<00:02, 177.41it/s]
 Epoch 601/1000, Loss: 0.0112
 67%| | 666/1000 [00:04<00:01, 167.85it/s]
 Epoch 651/1000, Loss: 0.0111
 72%| | 724/1000 [00:04<00:01, 179.32it/s]
 Epoch 701/1000, Loss: 0.0111
 78%| | 778/1000 [00:05<00:01, 162.14it/s]
 Epoch 751/1000, Loss: 0.0110
 83%| | 830/1000 [00:05<00:01, 167.27it/s]
 Epoch 801/1000, Loss: 0.0110
 88%| | 884/1000 [00:05<00:00, 173.53it/s]
 Epoch 851/1000, Loss: 0.0109
 92%| | 920/1000 [00:06<00:00, 170.86it/s]
 Epoch 901/1000, Loss: 0.0109
 97%| | 974/1000 [00:06<00:00, 161.96it/s]
 Epoch 951/1000, Loss: 0.0109
 100%| | 1000/1000 [00:06<00:00, 153.35it/s]
 Epoch 1001/1000, Loss: 0.0108

```
[30]: y_pred_tr = model(torch.from_numpy(X_train).float().to(device))
      y_hat_tr = y_pred_tr.detach().cpu().numpy()
```

```

orig_y_eval = y_train*(_max-_min) + _min
orig_y_hat = y_hat_tr*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

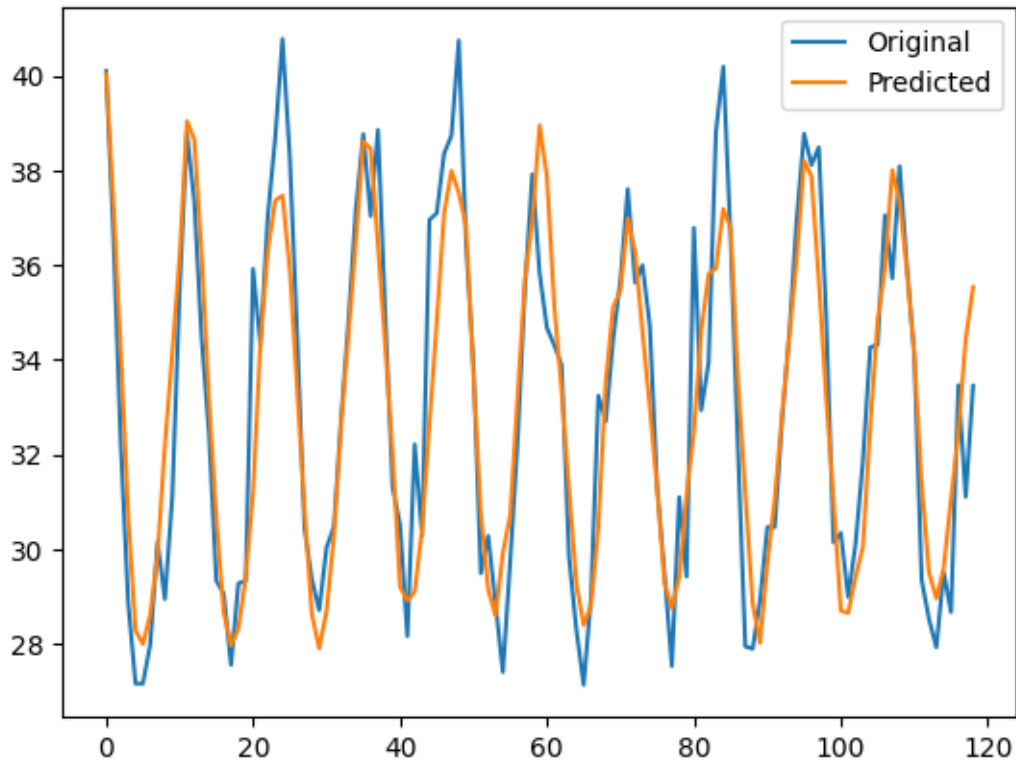
plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

Erro MSE = 2.659

Erro MAE = 1.254



```

[31]: y_pred = model(torch.from_numpy(X_test).float().to(device))
      y_hat = y_pred.detach().cpu().numpy()

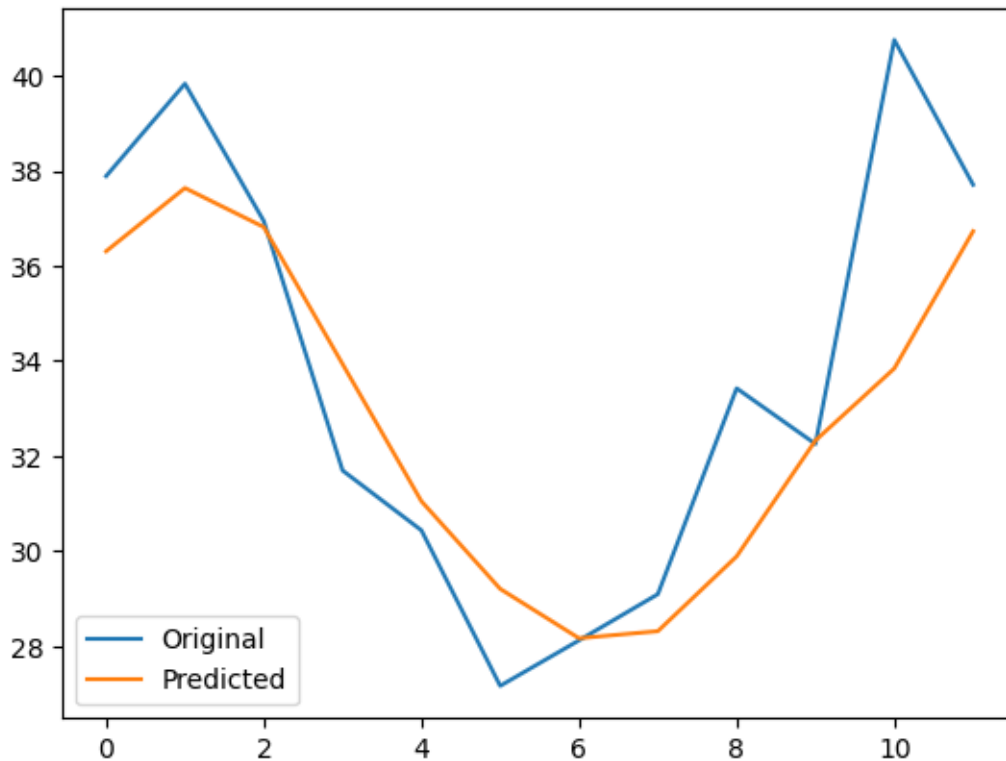
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_hat*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

```

```
plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')
```

Erro MSE = 6.558
 Erro MAE = 1.759



Podemos ver que a codificação mensal numérica é uma modelagem melhor do que a binária, já que o erro de teste foi menor que na questão anterior. Apesar do erro no treino ter sido ligeiramente maior, a curva da série predita em ambos os conjuntos de treino e teste, se apresentaram um pouco mais suavizadas, mostrando que generalizaram mais e ‘overfitaram’ menos.

1.6 Parte 3 - Previsão multi-step

Total: 5,0 pts.

1. Esquematize como você implementaria o processo de previsão multi-step. Descreva todos os passos necessários, e em seguida compare com o código fornecido para a previsão.

```
[ ]: def multi_step(model, X_test, cod='numeric', device='cpu', prev=12):
    y_result = []
    X_t = np.copy(X_test[0])
    print(X_t[0])
    for i in range(prev):
        y_hat = model(torch.from_numpy(X_t).float().to(device))

        if cod == 'numeric':

            #Ajeitando a codificação. Transformamos para o valor do mês e somamos 1
            ↪(para o próximo passo) e
            #depois pegamos o resto da divisão por 12 (para que o mês fique sempre
            ↪entre 0 e 11)
            tmp = int(X_t[0] * 12 + 1) % 12

            #Alterando a variável mês
            X_t[0] = tmp/12

            #Movendo todos os registros, 'deslizando a janela'
            X_t[1:X_t.shape[0]-1] = X_t[2:]

        elif cod == 'binary':
            pass

        else:
            print('Erro de codificação.')
            break

    X_t[-1] = y_hat

    y_result.append(y_hat[0].cpu().detach().numpy())
    return np.array(y_result)
```

Para fazer a previsão multistep prevendo o próximo estado utilizando uma variável exógena numérica contendo a informação do mês começando com 0, é necessário:

- 1 - Concatenar a variável exógena a janela de entrada. Nesse caso iremos botar como a primeira entrada do vetor.
- 2 - Fazer a previsão do próximo estado com o modelo.
- 3 - Agora deslocamos a janela de entrada de um mês e adicionamos nossa nova previsão ao final da janela de entrada.
- 4 - Também atualizamos nossa variável exógena já que queremos prever o próximo mês. Sendo assim primeiro iremos modificar a variável exógena. A modelagem é feita de forma que cada mês tenha o valor de 1/12 começando em 0 e indo até 11/12. Sendo assim incrementamos 1/12 e atualizamos a primeira entrada da janela de entrada.
- 5 - repetir os passos 2 ao 4 até terminar o loop.

2. Faça a previsão multi-step para o horizonte de previsão igual a 12. Perceba que o resultado da métrica é, geralmente, pior do que o apresentado para a previsão one-step. Por quê? É justo comparar o resultado deste item com o que foi apresentado anteriormente? Discuta.

```
[22]: def multi_step(model, X_test, cod='numeric', device='cpu', prev=12):
    y_result = []
    X_t = np.copy(X_test[0])
    for i in range(prev):
        y_hat = model(torch.from_numpy(X_t).float().to(device))

        if cod == 'numeric':

            #Ajeitando a codificação. Transformamos para o valor do mês e somamos 1
            ↪(para o próximo passo) e
            #depois pegamos o resto da divisão por 12 (para que o mês fique sempre
            ↪entre 0 e 11)
            tmp = int(X_t[0] * 12 + 1) % 12

            #Alterando a variável mês
            X_t[0] = tmp/12

            #Movendo todos os registros, 'deslizando a janela'
            X_t[1:X_t.shape[0]-1] = X_t[2:]

        elif cod == 'binary':
            pass

        else:
            print('Erro de codificação.')
            break

        X_t[-1] = y_hat

        y_result.append(y_hat[0].cpu().detach().numpy())
    return np.array(y_result)
```

```
[23]: prev = 12
train_val ,test_val = transform_data(raw_data, lag=12, test_split=prev,
    ↪normalize=True, category='numeric')
```

```
[24]: print(train_val.head(10))
```

	month	y(t-12)	y(t-11)	y(t-10)	y(t-9)	y(t-8)	y(t-7)	\
0	0.090909	0.980330	1.000000	0.834666	0.572063	0.162272	0.032500	
1	0.181818	1.000000	0.834666	0.572063	0.162272	0.032500	0.088908	
2	0.272727	0.834666	0.572063	0.162272	0.032500	0.088908	0.294752	

3	0.363636	0.572063	0.162272	0.032500	0.088908	0.294752	0.311236
4	0.454545	0.162272	0.032500	0.088908	0.294752	0.311236	0.281674
5	0.545455	0.032500	0.088908	0.294752	0.311236	0.281674	0.298381
6	0.636364	0.088908	0.294752	0.311236	0.281674	0.298381	0.771084
7	0.727273	0.294752	0.311236	0.281674	0.298381	0.771084	0.911432
8	0.818182	0.311236	0.281674	0.298381	0.771084	0.911432	0.854621
9	0.909091	0.281674	0.298381	0.771084	0.911432	0.854621	0.622018

	y(t-6)	y(t-5)	y(t-4)	y(t-3)	y(t-2)	y(t-1)	y(t) \
0	0.088908	0.294752	0.311236	0.281674	0.298381	0.771084	0.911432
1	0.294752	0.311236	0.281674	0.298381	0.771084	0.911432	0.854621
2	0.311236	0.281674	0.298381	0.771084	0.911432	0.854621	0.622018
3	0.281674	0.298381	0.771084	0.911432	0.854621	0.622018	0.333649
4	0.298381	0.771084	0.911432	0.854621	0.622018	0.333649	0.114426
5	0.771084	0.911432	0.854621	0.622018	0.333649	0.114426	0.001846
6	0.911432	0.854621	0.622018	0.333649	0.114426	0.001846	0.001722
7	0.854621	0.622018	0.333649	0.114426	0.001846	0.001722	0.055841
8	0.622018	0.333649	0.114426	0.001846	0.001722	0.055841	0.199390
9	0.333649	0.114426	0.001846	0.001722	0.055841	0.199390	0.119530

	y(t+1)
0	0.854621
1	0.622018
2	0.333649
3	0.114426
4	0.001846
5	0.001722
6	0.055841
7	0.199390
8	0.119530
9	0.261355

```
[25]: train_values = train_val.values
```

```
X_train = train_values[:, :-1]
y_train = train_values[:, -1]

test_values = test_val.values
X_test = test_values[:, :-1]
y_test = test_values[:, -1]
```

```
[26]: print(X_train[0])
print(y_train[0])
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```

[0.09090909 0.98033003 1.          0.83466593 0.57206297 0.16227193
 0.03249975 0.08890826 0.29475184 0.31123563 0.28167397 0.29838104
 0.77108408 0.91143197]
0.8546206961498539
(119, 14)
(119,)
(12, 14)
(12,)

```

```

[27]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 20

model = NeuralNetwork(n_input = n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000

history, y_hat = train(model, train_loader, epochs, device, lr)

```

cuda

```

0%|          | 0/1000 [00:00<?, ?it/s]  8%|          | 80/1000 [00:00<00:05,
156.86it/s]

```

Epoch 51/1000, Loss: 0.0210

```

12%|          | 116/1000 [00:00<00:05, 166.91it/s]

```

Epoch 101/1000, Loss: 0.0125

```

17%|          | 169/1000 [00:01<00:05, 164.18it/s]

```

Epoch 151/1000, Loss: 0.0107

```

22%|          | 222/1000 [00:01<00:05, 154.42it/s]

```

Epoch 201/1000, Loss: 0.0106

```

27%|          | 273/1000 [00:01<00:04, 161.22it/s]

```

Epoch 251/1000, Loss: 0.0108
32%| | 324/1000 [00:02<00:04, 152.65it/s]
Epoch 301/1000, Loss: 0.0108
37%| | 374/1000 [00:02<00:03, 158.63it/s]
Epoch 351/1000, Loss: 0.0108
43%| | 426/1000 [00:02<00:03, 164.86it/s]
Epoch 401/1000, Loss: 0.0108
48%| | 479/1000 [00:03<00:03, 161.35it/s]
Epoch 451/1000, Loss: 0.0108
53%| | 531/1000 [00:03<00:02, 162.97it/s]
Epoch 501/1000, Loss: 0.0107
56%| | 565/1000 [00:03<00:02, 159.60it/s]
Epoch 551/1000, Loss: 0.0107
62%| | 617/1000 [00:03<00:02, 161.82it/s]
Epoch 601/1000, Loss: 0.0107
67%| | 669/1000 [00:04<00:02, 160.42it/s]
Epoch 651/1000, Loss: 0.0106
72%| | 719/1000 [00:04<00:01, 156.18it/s]
Epoch 701/1000, Loss: 0.0106
78%| | 776/1000 [00:04<00:01, 179.29it/s]
Epoch 751/1000, Loss: 0.0106
83%| | 833/1000 [00:05<00:00, 183.72it/s]
Epoch 801/1000, Loss: 0.0105
87%| | 871/1000 [00:05<00:00, 179.91it/s]
Epoch 851/1000, Loss: 0.0105
93%| | 931/1000 [00:05<00:00, 192.70it/s]
Epoch 901/1000, Loss: 0.0105
97%| | 969/1000 [00:05<00:00, 170.37it/s]
Epoch 951/1000, Loss: 0.0105
100%| | 1000/1000 [00:06<00:00, 164.06it/s]
Epoch 1001/1000, Loss: 0.0104

```
[28]: y_result = multi_step(model, X_test, cod='numeric', device=device)
```

```
[29]: y_result
      print(y_result.shape)
      print(y_test.shape)
```

(12,)

(12,)

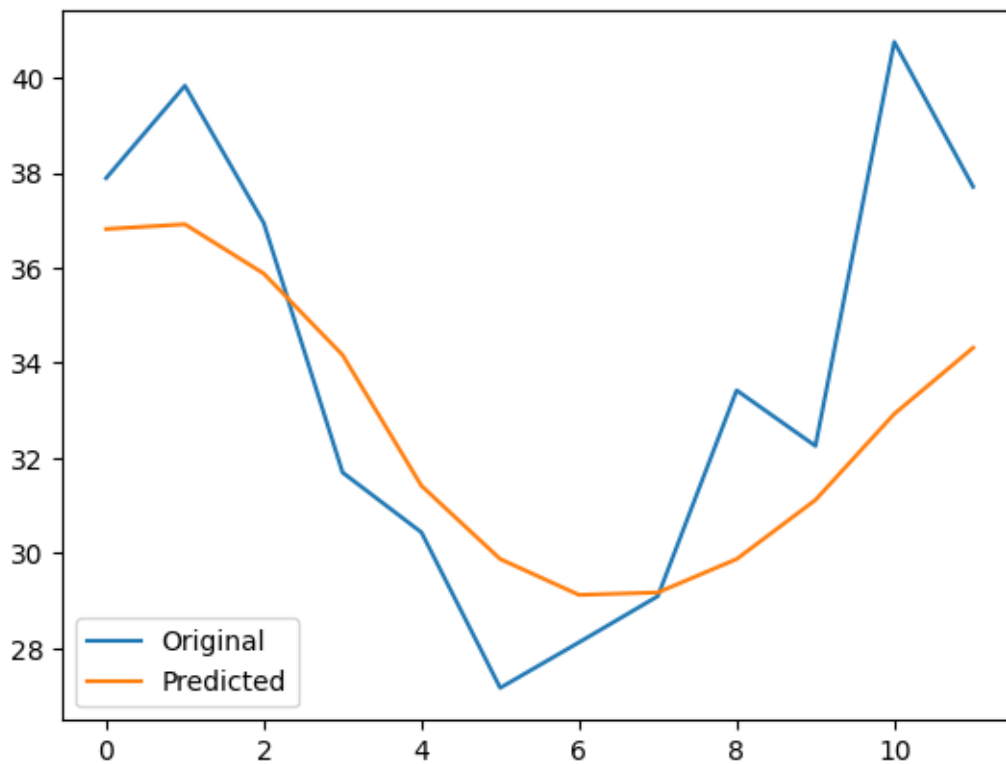
```
[30]: orig_y_eval = y_test*(_max-_min) + _min
      orig_y_hat = y_result*(_max-_min) + _min
      mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
      mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

      plt.plot(orig_y_eval)
      plt.plot(orig_y_hat)
      plt.legend(['Original', 'Predicted'])

      print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')
```

Erro MSE = 9.397

Erro MAE = 2.349



Podemos ver que o erro aumentou. Isso era esperado visto que o modelo foi treinado para prever apenas o mês seguinte e utilizamos ele de maneira autoregressiva para prever os 12. Isso faz com que o erro se acumule ao longo das previsões, aumentando o valor dele como acabamos de medir. Na questão anterior deslocamos a janela sem adicionar a previsão feita pelo modelo ao final dela, fazendo com o que o erro seja menor.

3. Modifique o tamanho da janela de entrada do modelo. Analise os resultados, utilizando as métricas adequadas.

```
[86]: def multi_step(model, X_test, cod='numeric', device='cpu', prev=12):
    y_result = []
    X_t = np.copy(X_test[0])
    for i in range(prev):
        y_hat = model(torch.from_numpy(X_t).float().to(device))

        if cod == 'numeric':

            #Ajeitando a codificação. Transformamos para o valor do mês e somamos 1
            ↪(para o próximo passo) e
            #depois pegamos o resto da divisão por 12 (para que o mês fique sempre
            ↪entre 0 e 11)
            tmp = int(X_t[0] * 12 + 1) % 12

            #Alterando a variável mês
            X_t[0] = tmp/12

            #Movendo todos os registros, 'deslizando a janela'
            X_t[1:X_t.shape[0]-1] = X_t[2:]

        elif cod == 'binary':
            pass

        else:
            print('Erro de codificação.')
            break

    X_t[-1] = y_hat

    y_result.append(y_hat[0].cpu().detach().numpy())
    return np.array(y_result)
```

```
[42]: prev = 12
train_val ,test_val = transform_data(raw_data, lag=12, test_split=prev,
    ↪normalize=True, category='numeric')
```

```
[57]: models = []
      for input_window in range(0, 12):
          train_values = train_val.values

          # input_window = 0
          X_train = train_values[:, input_window:-1]
          y_train = train_values[:, -1]

          train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
          ↪from_numpy(y_train).float().unsqueeze(1))
          test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
          ↪from_numpy(y_test).float().unsqueeze(1))

          batch_size = 32
          train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
          test_loader = DataLoader(dataset=test_dataset, batch_size=1)

          n_input = X_train.shape[1]
          n_output = 1
          hidden_neurons = 20

          model = NeuralNetwork(n_input = n_input, n_output=n_output,
          ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid')
          device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
          model.to(device)
          print(device)
          lr = 0.05
          epochs = 1000

          history, y_hat = train(model, train_loader, epochs, device, lr)
          models.append(model)
```

cuda

0%| | 0/1000 [00:00<?, ?it/s] 9%| | 87/1000 [00:00<00:05, 171.11it/s]

Epoch 51/1000, Loss: 0.0239

12%| | 122/1000 [00:00<00:05, 157.13it/s]

Epoch 101/1000, Loss: 0.0139

18%| | 177/1000 [00:01<00:04, 174.07it/s]

Epoch 151/1000, Loss: 0.0118

23%| | 230/1000 [00:01<00:04, 167.13it/s]

Epoch 201/1000, Loss: 0.0118

29%| | 287/1000 [00:01<00:03, 181.28it/s]

Epoch 251/1000, Loss: 0.0119
 32%| | 325/1000 [00:01<00:04, 165.88it/s]
 Epoch 301/1000, Loss: 0.0119
 38%| | 376/1000 [00:02<00:03, 163.05it/s]
 Epoch 351/1000, Loss: 0.0119
 43%| | 429/1000 [00:02<00:03, 169.13it/s]
 Epoch 401/1000, Loss: 0.0118
 48%| | 481/1000 [00:02<00:03, 164.19it/s]
 Epoch 451/1000, Loss: 0.0117
 52%| | 515/1000 [00:03<00:03, 160.28it/s]
 Epoch 501/1000, Loss: 0.0117
 58%| | 580/1000 [00:03<00:02, 151.69it/s]
 Epoch 551/1000, Loss: 0.0116
 63%| | 627/1000 [00:03<00:02, 145.23it/s]
 Epoch 601/1000, Loss: 0.0116
 68%| | 677/1000 [00:04<00:02, 147.94it/s]
 Epoch 651/1000, Loss: 0.0115
 73%| | 731/1000 [00:04<00:01, 166.64it/s]
 Epoch 701/1000, Loss: 0.0114
 79%| | 786/1000 [00:04<00:01, 174.25it/s]
 Epoch 751/1000, Loss: 0.0114
 82%| | 820/1000 [00:05<00:01, 145.51it/s]
 Epoch 801/1000, Loss: 0.0113
 87%| | 866/1000 [00:05<00:00, 135.28it/s]
 Epoch 851/1000, Loss: 0.0113
 92%| | 923/1000 [00:05<00:00, 132.41it/s]
 Epoch 901/1000, Loss: 0.0112
 97%| | 969/1000 [00:06<00:00, 144.19it/s]
 Epoch 951/1000, Loss: 0.0112
 100%| | 1000/1000 [00:06<00:00, 153.64it/s]
 Epoch 1001/1000, Loss: 0.0111
 cuda

8%| | 76/1000 [00:00<00:07, 130.53it/s]
 Epoch 51/1000, Loss: 0.0264
 13%| | 132/1000 [00:01<00:05, 168.86it/s]
 Epoch 101/1000, Loss: 0.0150
 17%| | 171/1000 [00:01<00:04, 179.84it/s]
 Epoch 151/1000, Loss: 0.0125
 23%| | 230/1000 [00:01<00:04, 176.87it/s]
 Epoch 201/1000, Loss: 0.0121
 27%| | 266/1000 [00:01<00:04, 154.51it/s]
 Epoch 251/1000, Loss: 0.0120
 32%| | 322/1000 [00:02<00:03, 174.29it/s]
 Epoch 301/1000, Loss: 0.0118
 38%| | 379/1000 [00:02<00:03, 177.12it/s]
 Epoch 351/1000, Loss: 0.0117
 43%| | 434/1000 [00:02<00:03, 176.52it/s]
 Epoch 401/1000, Loss: 0.0116
 47%| | 474/1000 [00:03<00:02, 185.78it/s]
 Epoch 451/1000, Loss: 0.0116
 53%| | 534/1000 [00:03<00:02, 176.65it/s]
 Epoch 501/1000, Loss: 0.0115
 57%| | 572/1000 [00:03<00:02, 167.68it/s]
 Epoch 551/1000, Loss: 0.0114
 62%| | 623/1000 [00:03<00:02, 148.99it/s]
 Epoch 601/1000, Loss: 0.0114
 68%| | 675/1000 [00:04<00:02, 159.32it/s]
 Epoch 651/1000, Loss: 0.0113
 71%| | 710/1000 [00:04<00:02, 143.10it/s]
 Epoch 701/1000, Loss: 0.0112
 78%| | 778/1000 [00:04<00:01, 146.98it/s]
 Epoch 751/1000, Loss: 0.0112
 83%| | 831/1000 [00:05<00:01, 165.66it/s]
 Epoch 801/1000, Loss: 0.0111

88%| | 884/1000 [00:05<00:00, 170.29it/s]
 Epoch 851/1000, Loss: 0.0111
 92%| | 919/1000 [00:05<00:00, 155.46it/s]
 Epoch 901/1000, Loss: 0.0111
 97%| | 969/1000 [00:06<00:00, 142.88it/s]
 Epoch 951/1000, Loss: 0.0110
 100%| | 1000/1000 [00:06<00:00, 156.28it/s]
 Epoch 1001/1000, Loss: 0.0110
 cuda
 7%| | 69/1000 [00:00<00:05, 159.01it/s]
 Epoch 51/1000, Loss: 0.0249
 12%| | 121/1000 [00:00<00:05, 162.04it/s]
 Epoch 101/1000, Loss: 0.0140
 17%| | 173/1000 [00:01<00:04, 165.64it/s]
 Epoch 151/1000, Loss: 0.0114
 23%| | 228/1000 [00:01<00:04, 161.44it/s]
 Epoch 201/1000, Loss: 0.0112
 28%| | 278/1000 [00:01<00:04, 154.10it/s]
 Epoch 251/1000, Loss: 0.0112
 33%| | 330/1000 [00:02<00:04, 163.16it/s]
 Epoch 301/1000, Loss: 0.0112
 38%| | 384/1000 [00:02<00:03, 172.07it/s]
 Epoch 351/1000, Loss: 0.0112
 42%| | 420/1000 [00:02<00:03, 155.08it/s]
 Epoch 401/1000, Loss: 0.0111
 48%| | 475/1000 [00:02<00:03, 173.03it/s]
 Epoch 451/1000, Loss: 0.0111
 53%| | 533/1000 [00:03<00:02, 184.43it/s]
 Epoch 501/1000, Loss: 0.0110
 57%| | 571/1000 [00:03<00:02, 179.75it/s]
 Epoch 551/1000, Loss: 0.0109
 63%| | 627/1000 [00:03<00:02, 179.80it/s]

Epoch 601/1000, Loss: 0.0109
68%| | 683/1000 [00:04<00:01, 174.55it/s]
Epoch 651/1000, Loss: 0.0108
72%| | 721/1000 [00:04<00:01, 180.48it/s]
Epoch 701/1000, Loss: 0.0108
78%| | 777/1000 [00:04<00:01, 175.37it/s]
Epoch 751/1000, Loss: 0.0107
83%| | 831/1000 [00:04<00:00, 170.40it/s]
Epoch 801/1000, Loss: 0.0107
87%| | 866/1000 [00:05<00:00, 140.02it/s]
Epoch 851/1000, Loss: 0.0106
92%| | 925/1000 [00:05<00:00, 125.67it/s]
Epoch 901/1000, Loss: 0.0106
96%| | 962/1000 [00:06<00:00, 108.63it/s]
Epoch 951/1000, Loss: 0.0106
100%| | 1000/1000 [00:06<00:00, 156.31it/s]
Epoch 1001/1000, Loss: 0.0105
cuda
8%| | 81/1000 [00:00<00:06, 153.02it/s]
Epoch 51/1000, Loss: 0.0298
13%| | 131/1000 [00:00<00:05, 148.42it/s]
Epoch 101/1000, Loss: 0.0195
18%| | 184/1000 [00:01<00:04, 163.39it/s]
Epoch 151/1000, Loss: 0.0156
22%| | 219/1000 [00:01<00:04, 166.13it/s]
Epoch 201/1000, Loss: 0.0147
27%| | 272/1000 [00:01<00:04, 171.34it/s]
Epoch 251/1000, Loss: 0.0144
33%| | 326/1000 [00:02<00:04, 154.41it/s]
Epoch 301/1000, Loss: 0.0142
38%| | 379/1000 [00:02<00:03, 165.60it/s]
Epoch 351/1000, Loss: 0.0140

43%| | 431/1000 [00:02<00:03, 157.68it/s]
Epoch 401/1000, Loss: 0.0138
48%| | 480/1000 [00:03<00:03, 153.00it/s]
Epoch 451/1000, Loss: 0.0136
51%| | 513/1000 [00:03<00:03, 146.85it/s]
Epoch 501/1000, Loss: 0.0134
58%| | 579/1000 [00:03<00:02, 155.63it/s]
Epoch 551/1000, Loss: 0.0132
63%| | 626/1000 [00:04<00:02, 149.08it/s]
Epoch 601/1000, Loss: 0.0131
68%| | 680/1000 [00:04<00:01, 168.81it/s]
Epoch 651/1000, Loss: 0.0129
72%| | 717/1000 [00:04<00:01, 174.62it/s]
Epoch 701/1000, Loss: 0.0128
77%| | 768/1000 [00:04<00:01, 154.99it/s]
Epoch 751/1000, Loss: 0.0127
82%| | 820/1000 [00:05<00:01, 161.93it/s]
Epoch 801/1000, Loss: 0.0125
87%| | 874/1000 [00:05<00:00, 167.70it/s]
Epoch 851/1000, Loss: 0.0124
93%| | 931/1000 [00:05<00:00, 181.36it/s]
Epoch 901/1000, Loss: 0.0123
97%| | 968/1000 [00:06<00:00, 166.08it/s]
Epoch 951/1000, Loss: 0.0122
100%| | 1000/1000 [00:06<00:00, 158.83it/s]
Epoch 1001/1000, Loss: 0.0121
cuda
8%| | 82/1000 [00:00<00:04, 201.09it/s]
Epoch 51/1000, Loss: 0.0392
12%| | 123/1000 [00:00<00:04, 193.87it/s]
Epoch 101/1000, Loss: 0.0242
18%| | 183/1000 [00:00<00:04, 190.50it/s]

Epoch 151/1000, Loss: 0.0157
 22%| | 220/1000 [00:01<00:05, 145.21it/s]
 Epoch 201/1000, Loss: 0.0131
 26%| | 264/1000 [00:01<00:05, 125.82it/s]
 Epoch 251/1000, Loss: 0.0126
 33%| | 329/1000 [00:02<00:04, 150.31it/s]
 Epoch 301/1000, Loss: 0.0126
 36%| | 361/1000 [00:02<00:05, 126.15it/s]
 Epoch 351/1000, Loss: 0.0126
 42%| | 419/1000 [00:02<00:04, 125.15it/s]
 Epoch 401/1000, Loss: 0.0125
 48%| | 478/1000 [00:03<00:03, 135.44it/s]
 Epoch 451/1000, Loss: 0.0125
 52%| | 524/1000 [00:03<00:03, 145.73it/s]
 Epoch 501/1000, Loss: 0.0124
 57%| | 572/1000 [00:03<00:02, 149.00it/s]
 Epoch 551/1000, Loss: 0.0123
 62%| | 624/1000 [00:04<00:02, 152.23it/s]
 Epoch 601/1000, Loss: 0.0122
 67%| | 674/1000 [00:04<00:02, 155.56it/s]
 Epoch 651/1000, Loss: 0.0122
 73%| | 728/1000 [00:04<00:01, 172.79it/s]
 Epoch 701/1000, Loss: 0.0121
 79%| | 786/1000 [00:05<00:01, 184.47it/s]
 Epoch 751/1000, Loss: 0.0121
 82%| | 824/1000 [00:05<00:01, 161.88it/s]
 Epoch 801/1000, Loss: 0.0120
 88%| | 879/1000 [00:05<00:00, 172.35it/s]
 Epoch 851/1000, Loss: 0.0120
 94%| | 938/1000 [00:06<00:00, 186.40it/s]
 Epoch 901/1000, Loss: 0.0119
 98%| | 976/1000 [00:06<00:00, 187.13it/s]

Epoch 951/1000, Loss: 0.0119
100%| | 1000/1000 [00:06<00:00, 155.76it/s]
Epoch 1001/1000, Loss: 0.0118
cuda
7%| | 73/1000 [00:00<00:04, 186.09it/s]
Epoch 51/1000, Loss: 0.0307
13%| | 134/1000 [00:00<00:04, 195.58it/s]
Epoch 101/1000, Loss: 0.0193
18%| | 175/1000 [00:00<00:04, 198.69it/s]
Epoch 151/1000, Loss: 0.0148
24%| | 238/1000 [00:01<00:03, 198.30it/s]
Epoch 201/1000, Loss: 0.0136
28%| | 278/1000 [00:01<00:04, 166.48it/s]
Epoch 251/1000, Loss: 0.0133
34%| | 340/1000 [00:01<00:03, 191.15it/s]
Epoch 301/1000, Loss: 0.0132
38%| | 380/1000 [00:02<00:03, 190.89it/s]
Epoch 351/1000, Loss: 0.0131
42%| | 420/1000 [00:02<00:03, 186.85it/s]
Epoch 401/1000, Loss: 0.0130
48%| | 481/1000 [00:02<00:02, 180.00it/s]
Epoch 451/1000, Loss: 0.0129
52%| | 519/1000 [00:02<00:02, 181.24it/s]
Epoch 501/1000, Loss: 0.0128
58%| | 578/1000 [00:03<00:02, 183.03it/s]
Epoch 551/1000, Loss: 0.0127
62%| | 615/1000 [00:03<00:02, 163.72it/s]
Epoch 601/1000, Loss: 0.0127
67%| | 669/1000 [00:03<00:01, 171.82it/s]
Epoch 651/1000, Loss: 0.0126
72%| | 723/1000 [00:04<00:01, 156.66it/s]
Epoch 701/1000, Loss: 0.0126

78%| | 776/1000 [00:04<00:01, 157.29it/s]
 Epoch 751/1000, Loss: 0.0125
 82%| | 824/1000 [00:04<00:01, 142.81it/s]
 Epoch 801/1000, Loss: 0.0124
 88%| | 875/1000 [00:05<00:00, 140.95it/s]
 Epoch 851/1000, Loss: 0.0124
 92%| | 923/1000 [00:05<00:00, 142.13it/s]
 Epoch 901/1000, Loss: 0.0124
 98%| | 980/1000 [00:05<00:00, 170.09it/s]
 Epoch 951/1000, Loss: 0.0123
 100%| | 1000/1000 [00:05<00:00, 171.02it/s]
 Epoch 1001/1000, Loss: 0.0123
 cuda
 7%| | 71/1000 [00:00<00:05, 176.06it/s]
 Epoch 51/1000, Loss: 0.0268
 13%| | 126/1000 [00:00<00:05, 158.99it/s]
 Epoch 101/1000, Loss: 0.0178
 18%| | 183/1000 [00:01<00:04, 176.54it/s]
 Epoch 151/1000, Loss: 0.0144
 22%| | 218/1000 [00:01<00:04, 162.03it/s]
 Epoch 201/1000, Loss: 0.0133
 27%| | 274/1000 [00:01<00:04, 173.29it/s]
 Epoch 251/1000, Loss: 0.0130
 33%| | 334/1000 [00:01<00:03, 176.72it/s]
 Epoch 301/1000, Loss: 0.0128
 37%| | 372/1000 [00:02<00:03, 182.35it/s]
 Epoch 351/1000, Loss: 0.0127
 43%| | 429/1000 [00:02<00:03, 177.32it/s]
 Epoch 401/1000, Loss: 0.0126
 46%| | 465/1000 [00:02<00:03, 161.39it/s]
 Epoch 451/1000, Loss: 0.0125
 53%| | 531/1000 [00:03<00:03, 153.42it/s]

Epoch 501/1000, Loss: 0.0124
 59%| | 587/1000 [00:03<00:02, 173.58it/s]
 Epoch 551/1000, Loss: 0.0123
 60%| | 605/1000 [00:03<00:02, 166.60it/s]
 Epoch 601/1000, Loss: 0.0122
 67%| | 669/1000 [00:04<00:02, 136.03it/s]
 Epoch 651/1000, Loss: 0.0121
 72%| | 717/1000 [00:04<00:02, 139.73it/s]
 Epoch 701/1000, Loss: 0.0120
 76%| | 764/1000 [00:04<00:01, 143.26it/s]
 Epoch 751/1000, Loss: 0.0120
 83%| | 826/1000 [00:05<00:01, 146.41it/s]
 Epoch 801/1000, Loss: 0.0119
 87%| | 873/1000 [00:05<00:00, 146.17it/s]
 Epoch 851/1000, Loss: 0.0118
 92%| | 919/1000 [00:05<00:00, 139.83it/s]
 Epoch 901/1000, Loss: 0.0117
 96%| | 960/1000 [00:06<00:00, 107.96it/s]
 Epoch 951/1000, Loss: 0.0117
 100%| | 1000/1000 [00:06<00:00, 150.68it/s]
 Epoch 1001/1000, Loss: 0.0116
 cuda
 7%| | 66/1000 [00:00<00:08, 110.38it/s]
 Epoch 51/1000, Loss: 0.0294
 12%| | 121/1000 [00:00<00:06, 133.66it/s]
 Epoch 101/1000, Loss: 0.0198
 18%| | 176/1000 [00:01<00:06, 121.19it/s]
 Epoch 151/1000, Loss: 0.0150
 21%| | 214/1000 [00:01<00:07, 109.37it/s]
 Epoch 201/1000, Loss: 0.0133
 27%| | 268/1000 [00:02<00:05, 126.97it/s]
 Epoch 251/1000, Loss: 0.0128

31%| | 312/1000 [00:02<00:05, 126.09it/s]
 Epoch 301/1000, Loss: 0.0126
 37%| | 369/1000 [00:03<00:04, 128.74it/s]
 Epoch 351/1000, Loss: 0.0126
 42%| | 421/1000 [00:03<00:03, 156.77it/s]
 Epoch 401/1000, Loss: 0.0125
 47%| | 474/1000 [00:03<00:03, 161.67it/s]
 Epoch 451/1000, Loss: 0.0124
 51%| | 507/1000 [00:03<00:03, 137.65it/s]
 Epoch 501/1000, Loss: 0.0123
 57%| | 567/1000 [00:04<00:03, 130.62it/s]
 Epoch 551/1000, Loss: 0.0122
 62%| | 625/1000 [00:04<00:02, 135.08it/s]
 Epoch 601/1000, Loss: 0.0122
 67%| | 674/1000 [00:05<00:02, 152.91it/s]
 Epoch 651/1000, Loss: 0.0121
 73%| | 730/1000 [00:05<00:01, 173.41it/s]
 Epoch 701/1000, Loss: 0.0120
 79%| | 786/1000 [00:05<00:01, 179.35it/s]
 Epoch 751/1000, Loss: 0.0119
 82%| | 822/1000 [00:06<00:01, 160.23it/s]
 Epoch 801/1000, Loss: 0.0119
 88%| | 877/1000 [00:06<00:00, 176.41it/s]
 Epoch 851/1000, Loss: 0.0118
 94%| | 936/1000 [00:06<00:00, 189.40it/s]
 Epoch 901/1000, Loss: 0.0117
 98%| | 975/1000 [00:06<00:00, 186.54it/s]
 Epoch 951/1000, Loss: 0.0117
 100%| | 1000/1000 [00:07<00:00, 142.71it/s]
 Epoch 1001/1000, Loss: 0.0116
 cuda
 7%| | 67/1000 [00:00<00:05, 173.53it/s]

Epoch 51/1000, Loss: 0.0316
 12%| | 121/1000 [00:00<00:05, 168.50it/s]
 Epoch 101/1000, Loss: 0.0217
 17%| | 172/1000 [00:01<00:04, 166.27it/s]
 Epoch 151/1000, Loss: 0.0163
 22%| | 223/1000 [00:01<00:04, 161.58it/s]
 Epoch 201/1000, Loss: 0.0142
 28%| | 276/1000 [00:01<00:04, 147.44it/s]
 Epoch 251/1000, Loss: 0.0135
 33%| | 328/1000 [00:02<00:04, 160.44it/s]
 Epoch 301/1000, Loss: 0.0132
 38%| | 380/1000 [00:02<00:03, 165.46it/s]
 Epoch 351/1000, Loss: 0.0130
 43%| | 433/1000 [00:02<00:03, 169.22it/s]
 Epoch 401/1000, Loss: 0.0129
 47%| | 467/1000 [00:02<00:03, 151.68it/s]
 Epoch 451/1000, Loss: 0.0128
 54%| | 536/1000 [00:03<00:02, 165.52it/s]
 Epoch 501/1000, Loss: 0.0127
 57%| | 570/1000 [00:03<00:02, 162.78it/s]
 Epoch 551/1000, Loss: 0.0126
 62%| | 624/1000 [00:03<00:02, 153.79it/s]
 Epoch 601/1000, Loss: 0.0125
 67%| | 672/1000 [00:04<00:02, 144.58it/s]
 Epoch 651/1000, Loss: 0.0125
 73%| | 726/1000 [00:04<00:01, 165.89it/s]
 Epoch 701/1000, Loss: 0.0124
 78%| | 782/1000 [00:04<00:01, 176.32it/s]
 Epoch 751/1000, Loss: 0.0123
 82%| | 818/1000 [00:05<00:01, 175.97it/s]
 Epoch 801/1000, Loss: 0.0123
 88%| | 875/1000 [00:05<00:00, 165.76it/s]

Epoch 851/1000, Loss: 0.0122
 93%| | 933/1000 [00:05<00:00, 181.87it/s]
 Epoch 901/1000, Loss: 0.0122
 97%| | 971/1000 [00:05<00:00, 180.00it/s]
 Epoch 951/1000, Loss: 0.0121
 100%| | 1000/1000 [00:06<00:00, 162.46it/s]
 Epoch 1001/1000, Loss: 0.0121
 cuda
 6%| | 64/1000 [00:00<00:04, 207.51it/s]
 Epoch 51/1000, Loss: 0.0312
 12%| | 123/1000 [00:00<00:04, 179.28it/s]
 Epoch 101/1000, Loss: 0.0217
 18%| | 180/1000 [00:00<00:04, 178.16it/s]
 Epoch 151/1000, Loss: 0.0157
 22%| | 218/1000 [00:01<00:04, 173.46it/s]
 Epoch 201/1000, Loss: 0.0129
 28%| | 275/1000 [00:01<00:04, 163.57it/s]
 Epoch 251/1000, Loss: 0.0118
 33%| | 330/1000 [00:01<00:03, 174.55it/s]
 Epoch 301/1000, Loss: 0.0114
 38%| | 384/1000 [00:02<00:03, 171.77it/s]
 Epoch 351/1000, Loss: 0.0113
 42%| | 420/1000 [00:02<00:03, 170.12it/s]
 Epoch 401/1000, Loss: 0.0112
 47%| | 474/1000 [00:02<00:03, 148.97it/s]
 Epoch 451/1000, Loss: 0.0112
 53%| | 526/1000 [00:03<00:02, 160.36it/s]
 Epoch 501/1000, Loss: 0.0111
 57%| | 575/1000 [00:03<00:02, 155.65it/s]
 Epoch 551/1000, Loss: 0.0111
 63%| | 626/1000 [00:03<00:02, 162.05it/s]
 Epoch 601/1000, Loss: 0.0111

68%| | 676/1000 [00:04<00:02, 151.98it/s]
Epoch 651/1000, Loss: 0.0110
73%| | 730/1000 [00:04<00:01, 167.30it/s]
Epoch 701/1000, Loss: 0.0110
76%| | 764/1000 [00:04<00:01, 147.35it/s]
Epoch 751/1000, Loss: 0.0110
84%| | 835/1000 [00:05<00:00, 168.67it/s]
Epoch 801/1000, Loss: 0.0110
87%| | 870/1000 [00:05<00:00, 143.83it/s]
Epoch 851/1000, Loss: 0.0109
92%| | 922/1000 [00:05<00:00, 156.31it/s]
Epoch 901/1000, Loss: 0.0109
98%| | 976/1000 [00:05<00:00, 167.62it/s]
Epoch 951/1000, Loss: 0.0109
100%| | 1000/1000 [00:06<00:00, 164.06it/s]
Epoch 1001/1000, Loss: 0.0109
cuda
8%| | 79/1000 [00:00<00:05, 158.46it/s]
Epoch 51/1000, Loss: 0.0427
14%| | 136/1000 [00:00<00:04, 177.72it/s]
Epoch 101/1000, Loss: 0.0354
17%| | 174/1000 [00:01<00:04, 180.06it/s]
Epoch 151/1000, Loss: 0.0287
23%| | 233/1000 [00:01<00:04, 184.58it/s]
Epoch 201/1000, Loss: 0.0229
27%| | 271/1000 [00:01<00:04, 156.23it/s]
Epoch 251/1000, Loss: 0.0185
33%| | 330/1000 [00:01<00:03, 178.26it/s]
Epoch 301/1000, Loss: 0.0157
39%| | 387/1000 [00:02<00:03, 181.68it/s]
Epoch 351/1000, Loss: 0.0140
43%| | 426/1000 [00:02<00:03, 185.03it/s]

Epoch 401/1000, Loss: 0.0132
 46%| | 464/1000 [00:02<00:03, 167.69it/s]
 Epoch 451/1000, Loss: 0.0128
 52%| | 520/1000 [00:03<00:02, 177.25it/s]
 Epoch 501/1000, Loss: 0.0126
 58%| | 579/1000 [00:03<00:02, 185.58it/s]
 Epoch 551/1000, Loss: 0.0124
 64%| | 637/1000 [00:03<00:01, 184.09it/s]
 Epoch 601/1000, Loss: 0.0124
 68%| | 675/1000 [00:03<00:01, 176.97it/s]
 Epoch 651/1000, Loss: 0.0123
 73%| | 729/1000 [00:04<00:01, 163.97it/s]
 Epoch 701/1000, Loss: 0.0122
 78%| | 782/1000 [00:04<00:01, 168.76it/s]
 Epoch 751/1000, Loss: 0.0122
 82%| | 816/1000 [00:04<00:01, 158.78it/s]
 Epoch 801/1000, Loss: 0.0121
 87%| | 869/1000 [00:05<00:00, 166.53it/s]
 Epoch 851/1000, Loss: 0.0121
 92%| | 922/1000 [00:05<00:00, 153.54it/s]
 Epoch 901/1000, Loss: 0.0121
 97%| | 973/1000 [00:05<00:00, 157.08it/s]
 Epoch 951/1000, Loss: 0.0120
 100%| | 1000/1000 [00:05<00:00, 168.39it/s]
 Epoch 1001/1000, Loss: 0.0120
 cuda
 7%| | 68/1000 [00:00<00:06, 147.28it/s]
 Epoch 51/1000, Loss: 0.0385
 11%| | 113/1000 [00:00<00:07, 126.00it/s]
 Epoch 101/1000, Loss: 0.0310
 18%| | 183/1000 [00:01<00:04, 163.47it/s]
 Epoch 151/1000, Loss: 0.0255

22%| | 217/1000 [00:01<00:05, 151.66it/s]
 Epoch 201/1000, Loss: 0.0215
 27%| | 270/1000 [00:01<00:04, 166.90it/s]
 Epoch 251/1000, Loss: 0.0185
 32%| | 323/1000 [00:02<00:04, 152.46it/s]
 Epoch 301/1000, Loss: 0.0163
 38%| | 377/1000 [00:02<00:03, 167.58it/s]
 Epoch 351/1000, Loss: 0.0147
 43%| | 433/1000 [00:02<00:03, 176.79it/s]
 Epoch 401/1000, Loss: 0.0136
 47%| | 471/1000 [00:02<00:02, 180.29it/s]
 Epoch 451/1000, Loss: 0.0129
 53%| | 529/1000 [00:03<00:02, 162.08it/s]
 Epoch 501/1000, Loss: 0.0125
 59%| | 588/1000 [00:03<00:02, 181.00it/s]
 Epoch 551/1000, Loss: 0.0122
 63%| | 626/1000 [00:03<00:02, 177.83it/s]
 Epoch 601/1000, Loss: 0.0121
 68%| | 682/1000 [00:04<00:01, 182.51it/s]
 Epoch 651/1000, Loss: 0.0120
 72%| | 720/1000 [00:04<00:01, 168.02it/s]
 Epoch 701/1000, Loss: 0.0119
 78%| | 775/1000 [00:04<00:01, 176.59it/s]
 Epoch 751/1000, Loss: 0.0119
 84%| | 835/1000 [00:05<00:00, 186.56it/s]
 Epoch 801/1000, Loss: 0.0119
 87%| | 874/1000 [00:05<00:00, 187.70it/s]
 Epoch 851/1000, Loss: 0.0119
 93%| | 931/1000 [00:05<00:00, 171.99it/s]
 Epoch 901/1000, Loss: 0.0119
 99%| | 986/1000 [00:05<00:00, 175.34it/s]
 Epoch 951/1000, Loss: 0.0119

100%| | 1000/1000 [00:06<00:00, 166.52it/s]

Epoch 1001/1000, Loss: 0.0119

```
[58]: y_results = []
      # y_tests = []
      for input_window in range(0, 12):
          test_values = test_val.values
          X_test = test_values[:, input_window:-1]
          y_test = test_values[:, -1]

          model = models[input_window]
          y_result = multi_step(model, X_test, cod='numeric', device=device)
          y_results.append(y_result)
          # y_tests.append(y_test)
```

```
[48]: print(y_result.shape)
      print(y_test.shape)
```

(12,)

(12,)

```
[59]: plt.figure(figsize=(10,6))

      orig_y_eval = y_test*(_max - _min) + _min
      # Plotando todas as séries em uma única figura
      for i, y_pred in enumerate(y_results):
          # Desnormaliza se necessário
          orig_y_hat = y_pred*(_max - _min) + _min

          plt.plot(orig_y_hat, label=f'Predicted {i}')

          mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
          mse_error = mean_squared_error(orig_y_eval, orig_y_hat)
          print(f'Input Window {i}: MSE = {round(mse_error,3)}, MAE = {
          ↪round(mae_error,3)}')

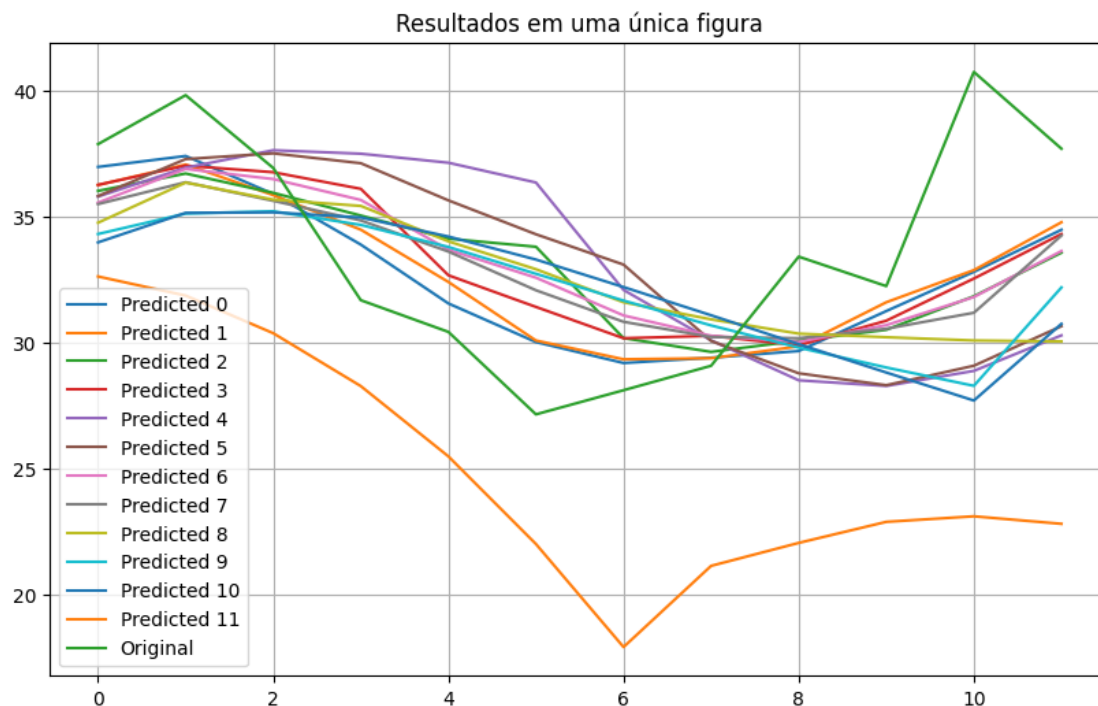
      # orig_y_eval = y_test*(_max - _min) + _min
      plt.plot(orig_y_eval, label=f'Original')

      plt.title('Resultados em uma única figura')
      plt.legend()
      plt.grid(True)
      plt.show()
```

Input Window 0: MSE = 9.302, MAE = 2.32

Input Window 1: MSE = 9.729, MAE = 2.473

Input Window 2: MSE = 16.516, MAE = 3.364
 Input Window 3: MSE = 12.649, MAE = 2.936
 Input Window 4: MSE = 35.73, MAE = 5.043
 Input Window 5: MSE = 30.583, MAE = 4.683
 Input Window 6: MSE = 15.859, MAE = 3.369
 Input Window 7: MSE = 15.741, MAE = 3.35
 Input Window 8: MSE = 23.702, MAE = 4.137
 Input Window 9: MSE = 26.101, MAE = 4.323
 Input Window 10: MSE = 30.48, MAE = 4.71
 Input Window 11: MSE = 92.674, MAE = 8.718



Fica claro que o erro aumenta consideravelmente conforme a janela de entrada diminui. Isso faz sentido, pois diminuimos a informação fornecida ao modelo. Quanto menos contexto damos ao modelo pior ele performa.

4. Modifique a topologia da rede para obter um melhor desempenho. Altere seus parâmetros (e.g. número de processadores na camada escondida, tipo de função na camada de saída) e avalie o desempenho.

```
[87]: prev = 12
      train_val ,test_val = transform_data(raw_data, lag=12, test_split=prev,
      ↪normalize=True, category='numeric')
```

```
[88]: train_values = train_val.values
```

```

input_window = 0
X_train = train_values[:,input_window:-1]
y_train = train_values[:, -1]

test_values = test_val.values
X_test = test_values[:,input_window:-1]
y_test = test_values[:, -1]

```

1.6.1 Hidden Neurons

```

[94]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 700

model = NeuralNetwork(n_input=n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid',
      ↪output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

cuda

7%| | 69/1000 [00:00<00:06, 138.65it/s]
 Epoch 51/1000, Loss: 0.0490
 12%| | 118/1000 [00:00<00:06, 133.39it/s]
 Epoch 101/1000, Loss: 0.0490
 16%| | 165/1000 [00:01<00:05, 143.65it/s]
 Epoch 151/1000, Loss: 0.0490
 24%| | 236/1000 [00:01<00:04, 167.83it/s]
 Epoch 201/1000, Loss: 0.0490
 27%| | 269/1000 [00:01<00:05, 144.50it/s]
 Epoch 251/1000, Loss: 0.0490
 32%| | 317/1000 [00:02<00:04, 153.69it/s]
 Epoch 301/1000, Loss: 0.0490
 37%| | 367/1000 [00:02<00:04, 148.52it/s]
 Epoch 351/1000, Loss: 0.0490
 42%| | 415/1000 [00:02<00:04, 141.21it/s]
 Epoch 401/1000, Loss: 0.0490
 48%| | 479/1000 [00:03<00:03, 153.25it/s]
 Epoch 451/1000, Loss: 0.0490
 53%| | 530/1000 [00:03<00:02, 162.46it/s]
 Epoch 501/1000, Loss: 0.0490
 58%| | 582/1000 [00:03<00:02, 162.84it/s]
 Epoch 551/1000, Loss: 0.0490
 62%| | 615/1000 [00:04<00:02, 152.40it/s]
 Epoch 601/1000, Loss: 0.0490
 68%| | 683/1000 [00:04<00:01, 160.95it/s]
 Epoch 651/1000, Loss: 0.0490
 72%| | 716/1000 [00:04<00:01, 150.68it/s]
 Epoch 701/1000, Loss: 0.0490
 76%| | 765/1000 [00:05<00:01, 135.40it/s]
 Epoch 751/1000, Loss: 0.0490
 83%| | 831/1000 [00:05<00:01, 153.65it/s]
 Epoch 801/1000, Loss: 0.0490

88%| | 880/1000 [00:05<00:00, 153.71it/s]

Epoch 851/1000, Loss: 0.0490

93%| | 930/1000 [00:06<00:00, 159.20it/s]

Epoch 901/1000, Loss: 0.0490

98%| | 978/1000 [00:06<00:00, 152.88it/s]

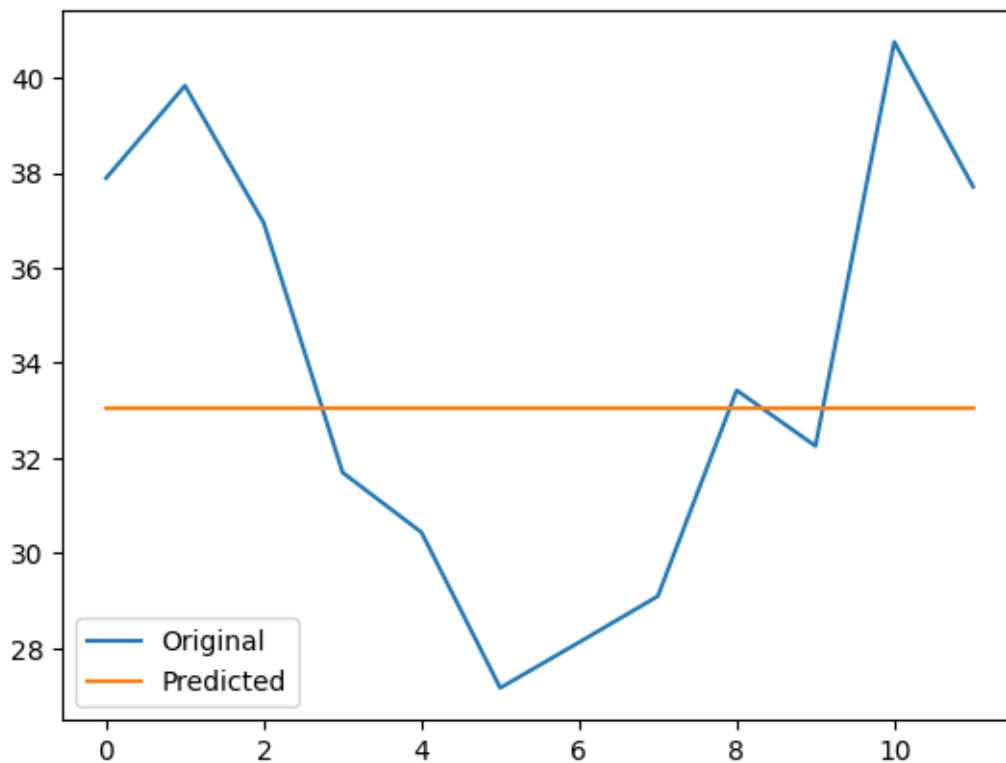
Epoch 951/1000, Loss: 0.0490

100%| | 1000/1000 [00:06<00:00, 149.60it/s]

Epoch 1001/1000, Loss: 0.0490

Erro MSE = 20.772

Erro MAE = 3.981



```
[92]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.  
      ↪from_numpy(y_train).float().unsqueeze(1))  
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.  
      ↪from_numpy(y_test).float().unsqueeze(1))  
  
batch_size = 32  
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)  
test_loader = DataLoader(dataset=test_dataset, batch_size=1)
```

```

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 70

model = NeuralNetwork(n_input=n_input, n_output=n_output,
    ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid',
    ↪output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

cuda

0%| | 0/1000 [00:00<?, ?it/s] 7%| | 70/1000 [00:00<00:07, 128.97it/s]

Epoch 51/1000, Loss: 0.0226

11%| | 113/1000 [00:00<00:06, 135.08it/s]

Epoch 101/1000, Loss: 0.0108

17%| | 172/1000 [00:01<00:05, 139.84it/s]

Epoch 151/1000, Loss: 0.0111

21%| | 214/1000 [00:01<00:06, 121.60it/s]

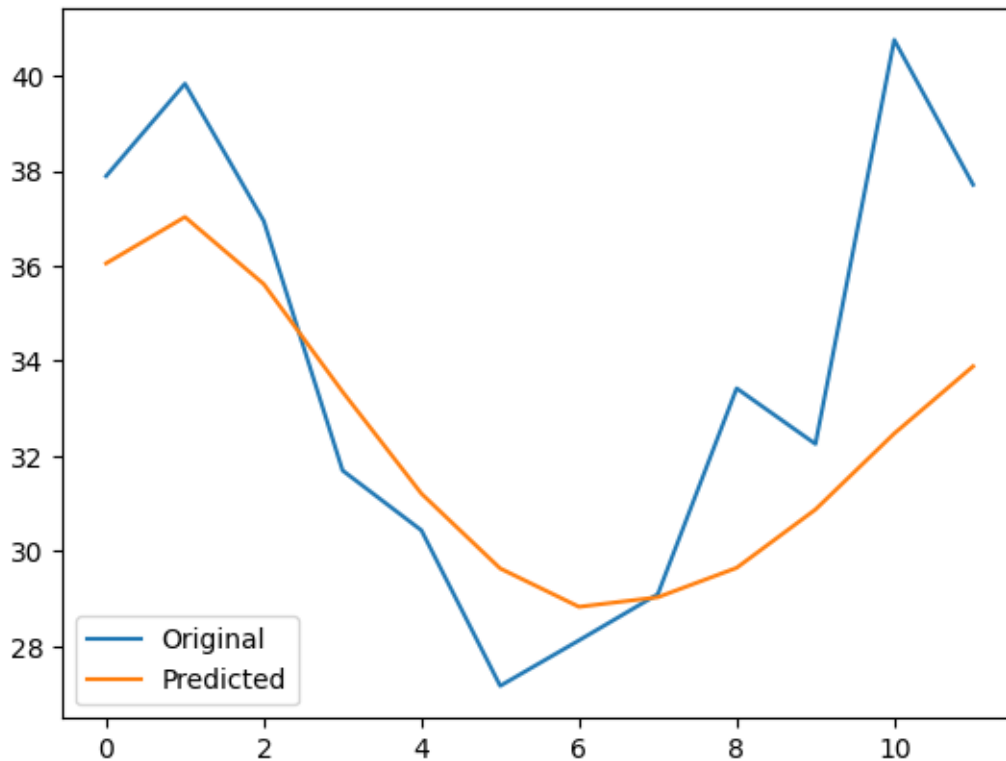
Epoch 201/1000, Loss: 0.0112

27%| | 268/1000 [00:02<00:05, 126.84it/s]

Epoch 251/1000, Loss: 0.0111

32%| | 318/1000 [00:02<00:04, 150.53it/s]

Epoch 301/1000, Loss: 0.0111
37%| | 367/1000 [00:02<00:04, 144.08it/s]
Epoch 351/1000, Loss: 0.0110
42%| | 415/1000 [00:03<00:03, 150.11it/s]
Epoch 401/1000, Loss: 0.0109
47%| | 471/1000 [00:03<00:04, 112.18it/s]
Epoch 451/1000, Loss: 0.0109
52%| | 520/1000 [00:04<00:04, 110.21it/s]
Epoch 501/1000, Loss: 0.0109
57%| | 571/1000 [00:04<00:03, 117.56it/s]
Epoch 551/1000, Loss: 0.0108
63%| | 626/1000 [00:04<00:02, 131.95it/s]
Epoch 601/1000, Loss: 0.0108
67%| | 674/1000 [00:05<00:02, 148.93it/s]
Epoch 651/1000, Loss: 0.0107
72%| | 722/1000 [00:05<00:01, 154.88it/s]
Epoch 701/1000, Loss: 0.0107
77%| | 768/1000 [00:05<00:01, 120.60it/s]
Epoch 751/1000, Loss: 0.0107
81%| | 814/1000 [00:06<00:01, 113.55it/s]
Epoch 801/1000, Loss: 0.0106
87%| | 870/1000 [00:06<00:00, 131.46it/s]
Epoch 851/1000, Loss: 0.0106
93%| | 926/1000 [00:07<00:00, 153.47it/s]
Epoch 901/1000, Loss: 0.0106
98%| | 978/1000 [00:07<00:00, 155.33it/s]
Epoch 951/1000, Loss: 0.0105
100%| | 1000/1000 [00:07<00:00, 132.07it/s]
Epoch 1001/1000, Loss: 0.0105
Erro MSE = 10.184
Erro MAE = 2.407



```
[93]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 35

model = NeuralNetwork(n_input=n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid',
      ↪output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000
```

```

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

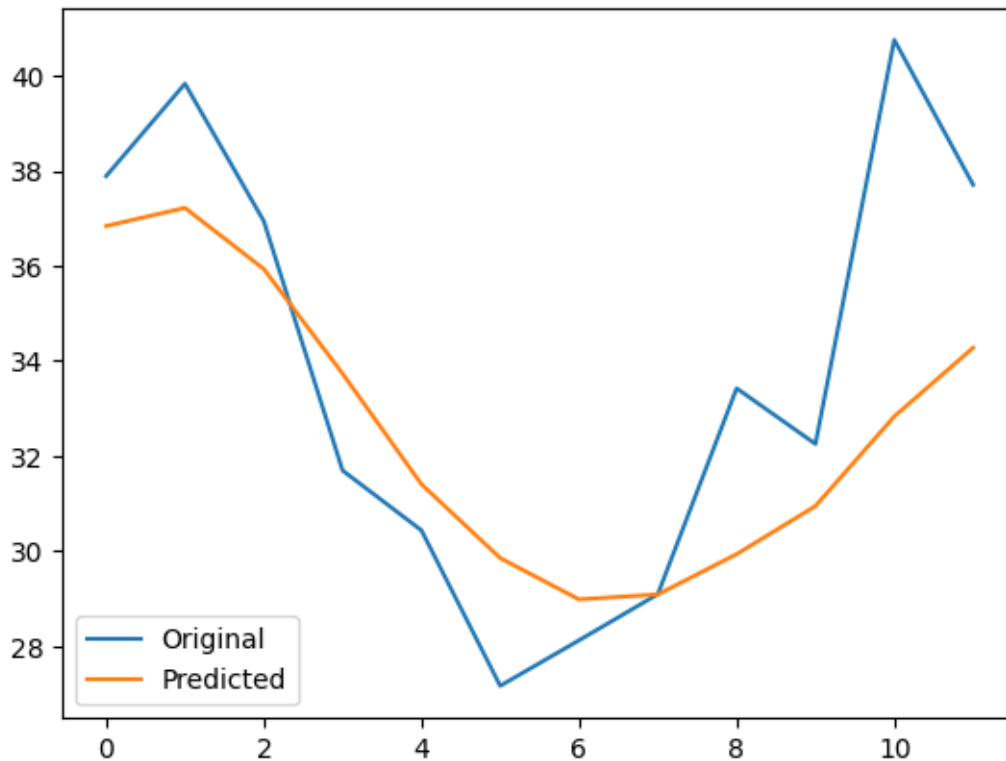
cuda

```

7%|          | 74/1000 [00:00<00:07, 127.98it/s]
Epoch 51/1000, Loss: 0.0189
14%|         | 135/1000 [00:00<00:04, 173.86it/s]
Epoch 101/1000, Loss: 0.0124
17%|         | 174/1000 [00:01<00:04, 182.37it/s]
Epoch 151/1000, Loss: 0.0118
24%|         | 235/1000 [00:01<00:03, 194.92it/s]
Epoch 201/1000, Loss: 0.0118
28%|         | 275/1000 [00:01<00:03, 188.46it/s]
Epoch 251/1000, Loss: 0.0119
31%|         | 313/1000 [00:01<00:04, 163.02it/s]
Epoch 301/1000, Loss: 0.0118
38%|         | 375/1000 [00:02<00:04, 134.39it/s]
Epoch 351/1000, Loss: 0.0117
42%|         | 423/1000 [00:02<00:03, 150.49it/s]
Epoch 401/1000, Loss: 0.0116
48%|         | 475/1000 [00:03<00:03, 161.87it/s]
Epoch 451/1000, Loss: 0.0115
53%|         | 527/1000 [00:03<00:02, 164.18it/s]
Epoch 501/1000, Loss: 0.0114
58%|         | 577/1000 [00:03<00:02, 151.37it/s]

```

Epoch 551/1000, Loss: 0.0114
62%| | 625/1000 [00:04<00:02, 150.78it/s]
Epoch 601/1000, Loss: 0.0113
67%| | 670/1000 [00:04<00:02, 127.54it/s]
Epoch 651/1000, Loss: 0.0112
72%| | 722/1000 [00:04<00:02, 123.70it/s]
Epoch 701/1000, Loss: 0.0112
76%| | 761/1000 [00:05<00:02, 115.19it/s]
Epoch 751/1000, Loss: 0.0111
82%| | 815/1000 [00:05<00:01, 127.34it/s]
Epoch 801/1000, Loss: 0.0110
88%| | 878/1000 [00:06<00:00, 147.86it/s]
Epoch 851/1000, Loss: 0.0110
92%| | 925/1000 [00:06<00:00, 151.89it/s]
Epoch 901/1000, Loss: 0.0109
97%| | 972/1000 [00:06<00:00, 133.95it/s]
Epoch 951/1000, Loss: 0.0109
100%| | 1000/1000 [00:07<00:00, 142.43it/s]
Epoch 1001/1000, Loss: 0.0108
Erro MSE = 9.198
Erro MAE = 2.281



```
[ ]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
    ↳from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
    ↳from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 10

model = NeuralNetwork(n_input=n_input, n_output=n_output,
    ↳hidden_neurons=hidden_neurons, hidden_activation='sigmoid',
    ↳output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000
```

```

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

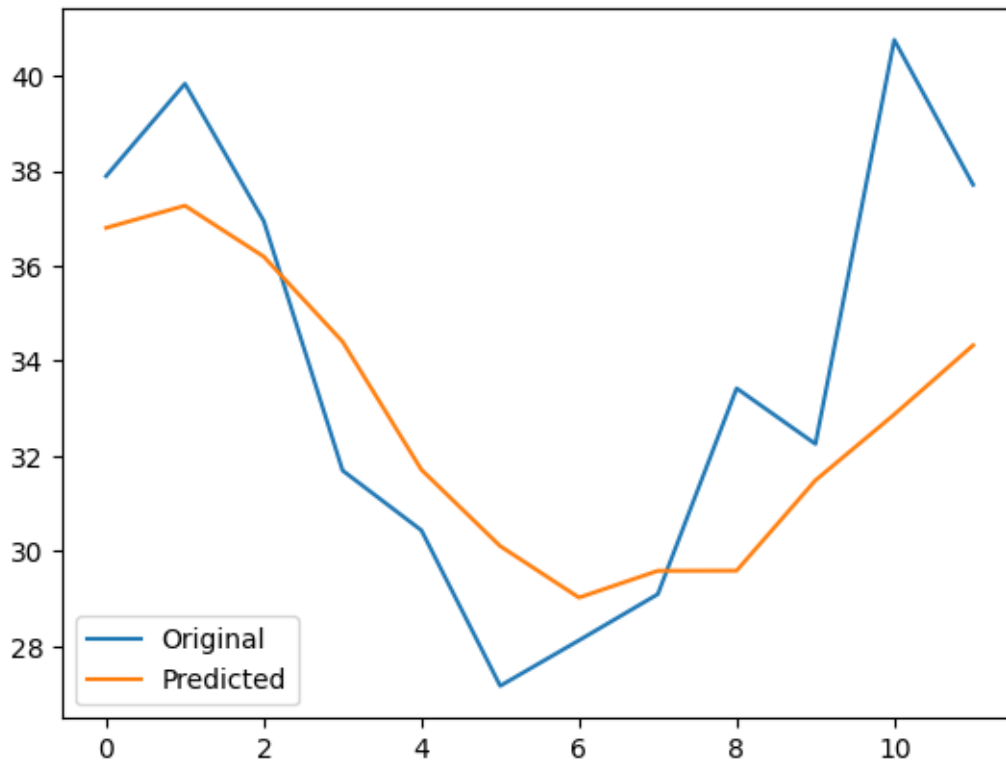
cuda

```

 8%|          | 82/1000 [00:00<00:05, 165.06it/s]
Epoch 51/1000, Loss: 0.0297
 13%|         | 132/1000 [00:00<00:05, 161.24it/s]
Epoch 101/1000, Loss: 0.0150
 16%|         | 165/1000 [00:01<00:05, 146.07it/s]
Epoch 151/1000, Loss: 0.0113
 21%|         | 213/1000 [00:01<00:05, 141.86it/s]
Epoch 201/1000, Loss: 0.0110
 28%|         | 279/1000 [00:01<00:04, 159.12it/s]
Epoch 251/1000, Loss: 0.0110
 33%|         | 331/1000 [00:02<00:04, 162.77it/s]
Epoch 301/1000, Loss: 0.0110
 36%|         | 363/1000 [00:02<00:04, 130.52it/s]
Epoch 351/1000, Loss: 0.0110
 42%|         | 417/1000 [00:02<00:03, 154.90it/s]
Epoch 401/1000, Loss: 0.0109
 47%|         | 470/1000 [00:03<00:03, 154.46it/s]
Epoch 451/1000, Loss: 0.0109
 50%|         | 505/1000 [00:03<00:03, 155.04it/s]
Epoch 501/1000, Loss: 0.0109
 55%|         | 554/1000 [00:04<00:08, 53.80it/s]

```

Epoch 551/1000, Loss: 0.0108
60%| | 605/1000 [00:05<00:08, 45.91it/s]
Epoch 601/1000, Loss: 0.0108
67%| | 668/1000 [00:06<00:02, 124.71it/s]
Epoch 651/1000, Loss: 0.0107
72%| | 719/1000 [00:06<00:01, 150.03it/s]
Epoch 701/1000, Loss: 0.0107
77%| | 767/1000 [00:06<00:01, 139.53it/s]
Epoch 751/1000, Loss: 0.0107
83%| | 826/1000 [00:07<00:01, 173.16it/s]
Epoch 801/1000, Loss: 0.0106
88%| | 885/1000 [00:07<00:00, 187.00it/s]
Epoch 851/1000, Loss: 0.0106
92%| | 925/1000 [00:07<00:00, 187.45it/s]
Epoch 901/1000, Loss: 0.0105
98%| | 985/1000 [00:07<00:00, 194.73it/s]
Epoch 951/1000, Loss: 0.0105
100%| | 1000/1000 [00:07<00:00, 125.89it/s]
Epoch 1001/1000, Loss: 0.0105
Erro MSE = 9.646
Erro MAE = 2.381



```
[78]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 20

model = NeuralNetwork(n_input=n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid',
      ↪output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000
```

```

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

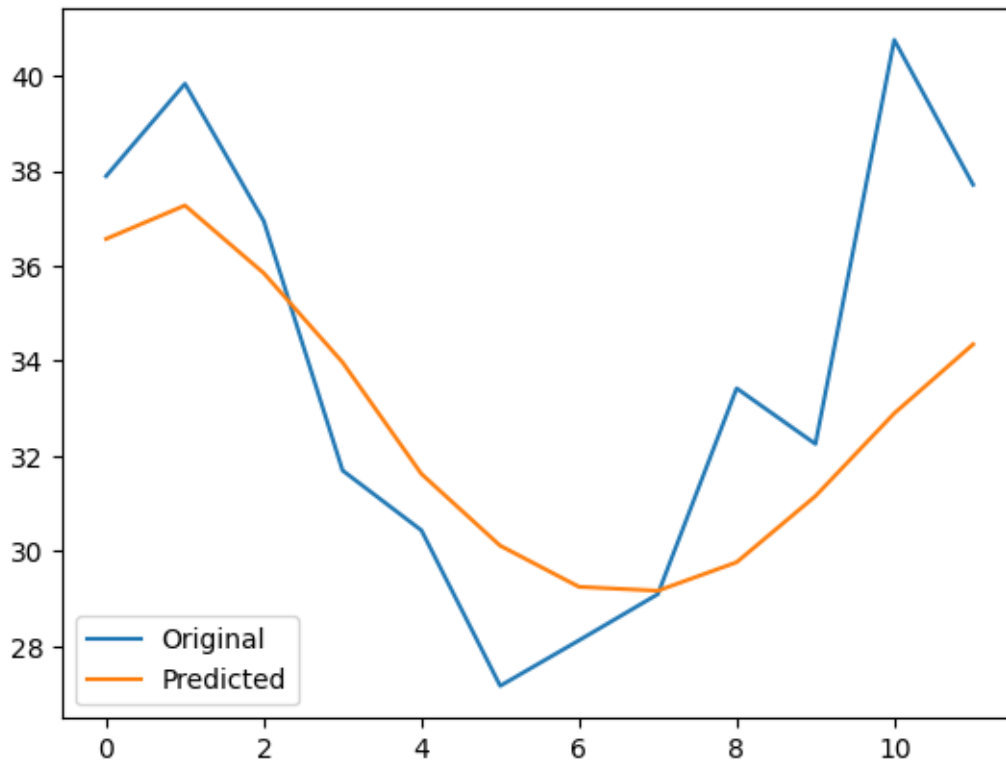
cuda

```

 9%|          | 88/1000 [00:00<00:05, 178.89it/s]
Epoch 51/1000, Loss: 0.0275
 12%|         | 125/1000 [00:00<00:04, 181.70it/s]
Epoch 101/1000, Loss: 0.0151
 18%|         | 180/1000 [00:01<00:04, 168.60it/s]
Epoch 151/1000, Loss: 0.0120
 23%|         | 234/1000 [00:01<00:04, 172.43it/s]
Epoch 201/1000, Loss: 0.0117
 27%|         | 271/1000 [00:01<00:04, 172.70it/s]
Epoch 251/1000, Loss: 0.0116
 33%|         | 328/1000 [00:01<00:04, 167.64it/s]
Epoch 301/1000, Loss: 0.0116
 38%|         | 377/1000 [00:02<00:04, 153.13it/s]
Epoch 351/1000, Loss: 0.0115
 43%|         | 426/1000 [00:02<00:03, 154.33it/s]
Epoch 401/1000, Loss: 0.0114
 47%|         | 471/1000 [00:02<00:04, 129.44it/s]
Epoch 451/1000, Loss: 0.0113
 53%|         | 530/1000 [00:03<00:03, 140.68it/s]
Epoch 501/1000, Loss: 0.0112
 58%|         | 576/1000 [00:03<00:02, 146.21it/s]

```

Epoch 551/1000, Loss: 0.0111
62%| | 625/1000 [00:04<00:02, 156.27it/s]
Epoch 601/1000, Loss: 0.0110
68%| | 676/1000 [00:04<00:02, 153.87it/s]
Epoch 651/1000, Loss: 0.0110
73%| | 726/1000 [00:04<00:01, 156.32it/s]
Epoch 701/1000, Loss: 0.0109
78%| | 777/1000 [00:05<00:01, 152.35it/s]
Epoch 751/1000, Loss: 0.0108
83%| | 830/1000 [00:05<00:01, 164.65it/s]
Epoch 801/1000, Loss: 0.0108
88%| | 882/1000 [00:05<00:00, 163.38it/s]
Epoch 851/1000, Loss: 0.0107
91%| | 914/1000 [00:05<00:00, 143.74it/s]
Epoch 901/1000, Loss: 0.0107
98%| | 985/1000 [00:06<00:00, 164.04it/s]
Epoch 951/1000, Loss: 0.0106
100%| | 1000/1000 [00:06<00:00, 155.09it/s]
Epoch 1001/1000, Loss: 0.0106
Erro MSE = 9.466
Erro MAE = 2.379



1.6.2 Funções de ativação

```
[95]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 20

model = NeuralNetwork(n_input=n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid',
      ↪output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
```

```

lr = 0.05
epochs = 1000

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')
```

cuda

2%| | 16/1000 [00:00<00:06, 151.49it/s] 8%| | 80/1000
[00:00<00:06, 146.00it/s]

Epoch 51/1000, Loss: 0.0216

13%| | 133/1000 [00:00<00:05, 161.40it/s]

Epoch 101/1000, Loss: 0.0130

17%| | 167/1000 [00:01<00:05, 154.89it/s]

Epoch 151/1000, Loss: 0.0116

23%| | 234/1000 [00:01<00:04, 162.21it/s]

Epoch 201/1000, Loss: 0.0116

27%| | 269/1000 [00:01<00:04, 150.83it/s]

Epoch 251/1000, Loss: 0.0116

31%| | 314/1000 [00:02<00:05, 132.25it/s]

Epoch 301/1000, Loss: 0.0115

37%| | 370/1000 [00:02<00:04, 132.49it/s]

Epoch 351/1000, Loss: 0.0115

43%| | 429/1000 [00:02<00:04, 141.72it/s]

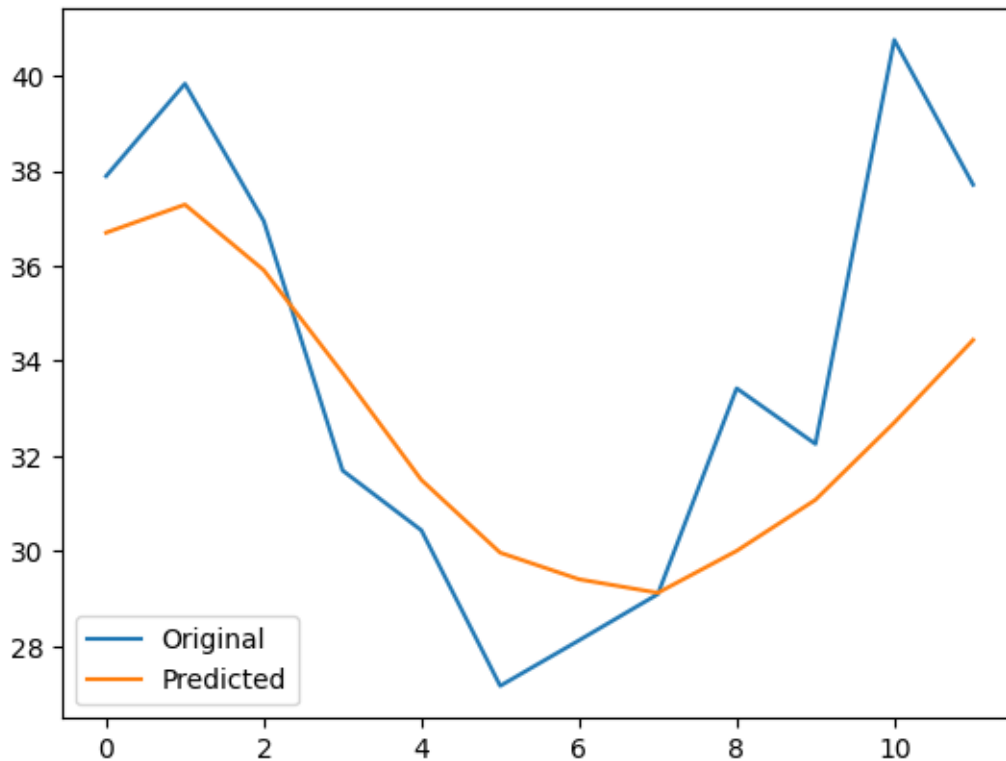
Epoch 401/1000, Loss: 0.0114

48%| | 475/1000 [00:03<00:03, 146.05it/s]

Epoch 451/1000, Loss: 0.0114

52%| | 518/1000 [00:03<00:03, 128.34it/s]

Epoch 501/1000, Loss: 0.0113
57%| | 574/1000 [00:04<00:03, 131.42it/s]
Epoch 551/1000, Loss: 0.0112
62%| | 616/1000 [00:04<00:02, 130.93it/s]
Epoch 601/1000, Loss: 0.0112
67%| | 674/1000 [00:04<00:02, 133.37it/s]
Epoch 651/1000, Loss: 0.0111
71%| | 714/1000 [00:05<00:02, 113.46it/s]
Epoch 701/1000, Loss: 0.0111
77%| | 768/1000 [00:05<00:01, 125.04it/s]
Epoch 751/1000, Loss: 0.0110
82%| | 821/1000 [00:06<00:01, 119.56it/s]
Epoch 801/1000, Loss: 0.0110
88%| | 877/1000 [00:06<00:00, 132.32it/s]
Epoch 851/1000, Loss: 0.0109
92%| | 925/1000 [00:06<00:00, 149.31it/s]
Epoch 901/1000, Loss: 0.0109
98%| | 979/1000 [00:07<00:00, 168.45it/s]
Epoch 951/1000, Loss: 0.0108
100%| | 1000/1000 [00:07<00:00, 138.10it/s]
Epoch 1001/1000, Loss: 0.0108
Erro MSE = 9.358
Erro MAE = 2.325



```
[97]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 20

model = NeuralNetwork(n_input=n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='relu',
      ↪output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000
```

```

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

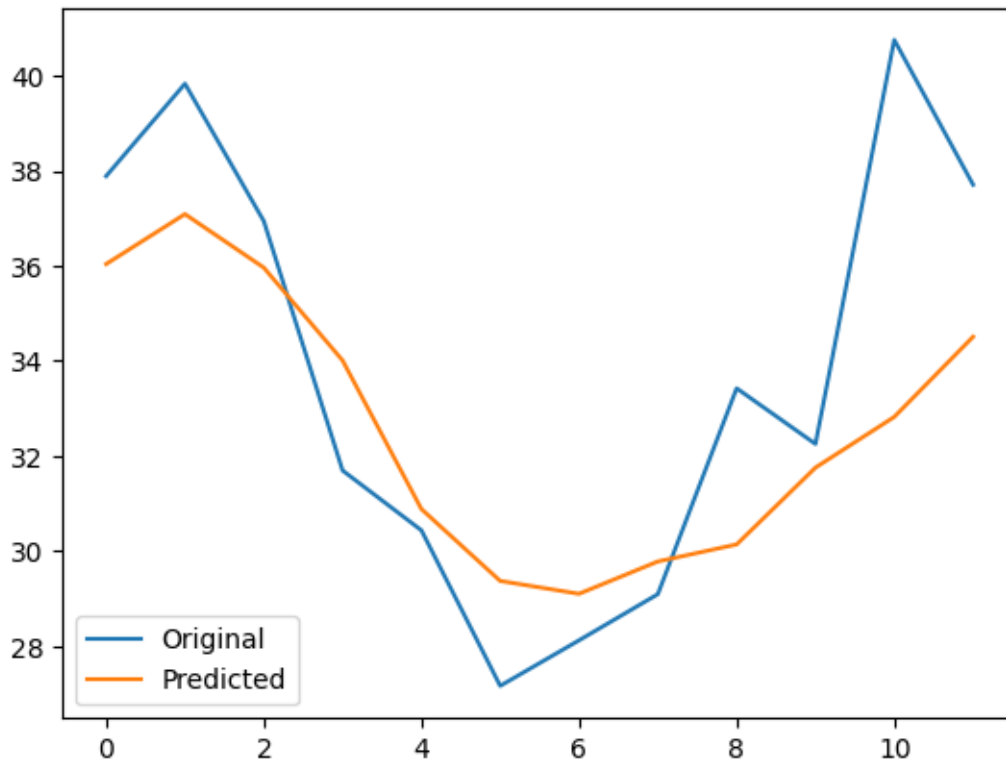
cuda

```

7%|          | 72/1000 [00:00<00:06, 134.63it/s]
Epoch 51/1000, Loss: 0.0102
12%|         | 123/1000 [00:00<00:05, 155.84it/s]
Epoch 101/1000, Loss: 0.0098
17%|         | 171/1000 [00:01<00:05, 151.19it/s]
Epoch 151/1000, Loss: 0.0095
22%|         | 223/1000 [00:01<00:04, 162.33it/s]
Epoch 201/1000, Loss: 0.0092
27%|         | 274/1000 [00:01<00:04, 162.22it/s]
Epoch 251/1000, Loss: 0.0090
32%|         | 324/1000 [00:02<00:04, 159.03it/s]
Epoch 301/1000, Loss: 0.0088
37%|         | 372/1000 [00:02<00:04, 150.11it/s]
Epoch 351/1000, Loss: 0.0087
42%|         | 418/1000 [00:02<00:04, 140.26it/s]
Epoch 401/1000, Loss: 0.0086
47%|         | 473/1000 [00:03<00:04, 126.24it/s]
Epoch 451/1000, Loss: 0.0085
52%|         | 523/1000 [00:03<00:03, 150.84it/s]
Epoch 501/1000, Loss: 0.0084
57%|         | 573/1000 [00:03<00:02, 158.09it/s]

```

Epoch 551/1000, Loss: 0.0083
62%| | 621/1000 [00:04<00:02, 147.07it/s]
Epoch 601/1000, Loss: 0.0082
67%| | 670/1000 [00:04<00:02, 151.79it/s]
Epoch 651/1000, Loss: 0.0081
72%| | 718/1000 [00:04<00:01, 146.99it/s]
Epoch 701/1000, Loss: 0.0080
78%| | 780/1000 [00:05<00:01, 147.40it/s]
Epoch 751/1000, Loss: 0.0079
83%| | 827/1000 [00:05<00:01, 147.06it/s]
Epoch 801/1000, Loss: 0.0078
87%| | 874/1000 [00:05<00:00, 152.73it/s]
Epoch 851/1000, Loss: 0.0077
92%| | 923/1000 [00:06<00:00, 152.45it/s]
Epoch 901/1000, Loss: 0.0076
97%| | 971/1000 [00:06<00:00, 147.54it/s]
Epoch 951/1000, Loss: 0.0075
100%| | 1000/1000 [00:06<00:00, 146.96it/s]
Epoch 1001/1000, Loss: 0.0074
Erro MSE = 8.992
Erro MAE = 2.26



```
[98]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
      ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
      ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 20

model = NeuralNetwork(n_input=n_input, n_output=n_output,
      ↪hidden_neurons=hidden_neurons, hidden_activation='relu',
      ↪output_activation='sigmoid')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000
```

```

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='numeric', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')

```

cuda

0%| | 0/1000 [00:00<?, ?it/s] 7%| | 69/1000 [00:00<00:06, 136.46it/s]

Epoch 51/1000, Loss: 0.0101

13%| | 134/1000 [00:00<00:05, 157.26it/s]

Epoch 101/1000, Loss: 0.0098

17%| | 166/1000 [00:01<00:05, 151.25it/s]

Epoch 151/1000, Loss: 0.0096

22%| | 217/1000 [00:01<00:04, 159.31it/s]

Epoch 201/1000, Loss: 0.0094

27%| | 269/1000 [00:01<00:04, 162.83it/s]

Epoch 251/1000, Loss: 0.0093

32%| | 321/1000 [00:02<00:04, 158.61it/s]

Epoch 301/1000, Loss: 0.0091

37%| | 372/1000 [00:02<00:03, 163.94it/s]

Epoch 351/1000, Loss: 0.0089

42%| | 423/1000 [00:02<00:03, 159.10it/s]

Epoch 401/1000, Loss: 0.0088

47%| | 471/1000 [00:03<00:03, 155.50it/s]

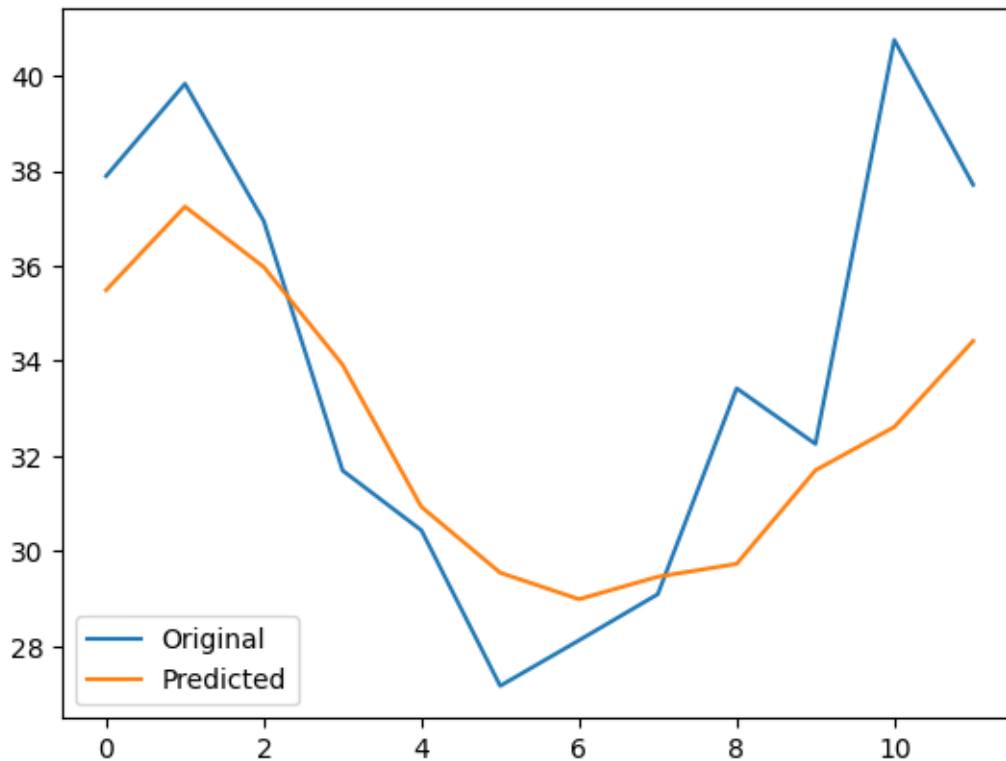
Epoch 451/1000, Loss: 0.0087

52%| | 521/1000 [00:03<00:02, 161.08it/s]

Epoch 501/1000, Loss: 0.0085

57%| | 573/1000 [00:03<00:02, 165.30it/s]

Epoch 551/1000, Loss: 0.0084
62%| | 625/1000 [00:04<00:02, 159.31it/s]
Epoch 601/1000, Loss: 0.0083
68%| | 676/1000 [00:04<00:02, 160.01it/s]
Epoch 651/1000, Loss: 0.0082
73%| | 728/1000 [00:04<00:01, 158.87it/s]
Epoch 701/1000, Loss: 0.0082
78%| | 778/1000 [00:05<00:01, 155.83it/s]
Epoch 751/1000, Loss: 0.0081
83%| | 826/1000 [00:05<00:01, 144.15it/s]
Epoch 801/1000, Loss: 0.0080
87%| | 871/1000 [00:05<00:00, 139.39it/s]
Epoch 851/1000, Loss: 0.0079
92%| | 918/1000 [00:06<00:00, 147.95it/s]
Epoch 901/1000, Loss: 0.0078
98%| | 981/1000 [00:06<00:00, 150.90it/s]
Epoch 951/1000, Loss: 0.0078
100%| | 1000/1000 [00:06<00:00, 151.80it/s]
Epoch 1001/1000, Loss: 0.0077
Erro MSE = 9.679
Erro MAE = 2.329



Podemos ver que conforme o número de neurônios na camada escondida aumenta o erro também aumenta. Isso ocorre devido ao fato dos neurônios decorarem o conjunto de treino, “overfit”, não generalizando bem no conjunto de teste. Entretanto, esse número precisa ser demasiadamente alto. Entre 1 e 100 neurônios o impacto é praticamente o mesmo.

Quanto as funções de ativação, comparei as funções de ativação utilizadas no exercício hidden=sigmoid e output=softmax com outras combinações. Podemos ver que ao mudar a função de ativação das camadas internas, todas exceto a de saída, pela “relu” temos uma ligeira melhora. A função de ativação “relu” é, no geral, a escolha mais adequada para os neurônios internos pois ela é a que melhor mimetiza o funcionamento dos neurônios biológicos. A intuição é que ela possui um valor de 0 até um certo limiar de ativação e então após isso deixa passar a informação na forma de uma função linear. Além disso, ao tentar mudar a função do neurônio de saída da softmax para sigmoid não tiveram mudanças expressivas.

5. Modifique a função de previsão multi-step para permitir a codificação binária da variável mês. Em seguida, treine um modelo MLP usando a codificação binária para a variável exógena e faça a previsão multi-step e analise o resultado. OBS: Para este item, basta alterar a atualização da variável exógena na condição desejada (no caso, cod = ‘binary’)

```
[121]: def multi_step(model, X_test, cod='numeric', device='cpu', prev=12):
        y_result = []
        X_t = np.copy(X_test[0])
        for i in range(prev):
```

```

y_hat = model(torch.from_numpy(X_t).float().to(device))

if cod == 'numeric':

    #Ajeitando a codificação. Transformamos para o valor do mês e somamos 1
    ↪(para o próximo passo) e
    #depois pegamos o resto da divisão por 12 (para que o mês fique sempre
    ↪entre 0 e 11)
    tmp = int(X_t[0] * 12 + 1) % 12

    #Alterando a variável mês
    X_t[0] = tmp/12

    #Movendo todos os registros, 'deslizando a janela'
    X_t[1:X_t.shape[0]-1] = X_t[2:]

elif cod == 'binary':
    # Supondo meses no intervalo 0-11
    # meses = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

    # Gera as strings binárias de 4 bits
    # binarios = [np.binary_repr(m, width=4) for m in meses]
    print(X_t[:4], i, int(X_t[0] + i))
    binario_str = np.binary_repr(int(X_t[0] + i), width=4)

    # Converte cada bit em uma coluna separada
    t0 = np.array(int(binario_str[0])).reshape(-1, 1)
    t1 = np.array(int(binario_str[1])).reshape(-1, 1)
    t2 = np.array(int(binario_str[2])).reshape(-1, 1)
    t3 = np.array(int(binario_str[3])).reshape(-1, 1)

    # Concatena para formar o array final
    cod_meses = np.hstack((t0, t1, t2, t3))
    # print(cod_meses)

    X_t[:4] = cod_meses # Atualiza os primeiros 4 elementos com a
    ↪codificação binária

else:
    print('Erro de codificação.')
    break

X_t[-1] = y_hat

y_result.append(y_hat[0].cpu().detach().numpy())
return np.array(y_result)

```

```
[123]: prev = 12
train_val ,test_val = transform_data(raw_data, lag=12, test_split=prev,
↪normalize=True, category='binary')
```

```
[124]: print(train_val.head(10))
```

	month_0	month_1	month_2	month_3	y(t-12)	y(t-11)	y(t-10)	y(t-9)	\
0	0.0	0.0	1.0	0.0	0.980330	1.000000	0.834666	0.572063	
1	0.0	0.0	1.0	1.0	1.000000	0.834666	0.572063	0.162272	
2	0.0	1.0	0.0	0.0	0.834666	0.572063	0.162272	0.032500	
3	0.0	1.0	0.0	1.0	0.572063	0.162272	0.032500	0.088908	
4	0.0	1.0	1.0	0.0	0.162272	0.032500	0.088908	0.294752	
5	0.0	1.0	1.0	1.0	0.032500	0.088908	0.294752	0.311236	
6	1.0	0.0	0.0	0.0	0.088908	0.294752	0.311236	0.281674	
7	1.0	0.0	0.0	1.0	0.294752	0.311236	0.281674	0.298381	
8	1.0	0.0	1.0	0.0	0.311236	0.281674	0.298381	0.771084	
9	1.0	0.0	1.0	1.0	0.281674	0.298381	0.771084	0.911432	

	y(t-8)	y(t-7)	y(t-6)	y(t-5)	y(t-4)	y(t-3)	y(t-2)	\
0	0.162272	0.032500	0.088908	0.294752	0.311236	0.281674	0.298381	
1	0.032500	0.088908	0.294752	0.311236	0.281674	0.298381	0.771084	
2	0.088908	0.294752	0.311236	0.281674	0.298381	0.771084	0.911432	
3	0.294752	0.311236	0.281674	0.298381	0.771084	0.911432	0.854621	
4	0.311236	0.281674	0.298381	0.771084	0.911432	0.854621	0.622018	
5	0.281674	0.298381	0.771084	0.911432	0.854621	0.622018	0.333649	
6	0.298381	0.771084	0.911432	0.854621	0.622018	0.333649	0.114426	
7	0.771084	0.911432	0.854621	0.622018	0.333649	0.114426	0.001846	
8	0.911432	0.854621	0.622018	0.333649	0.114426	0.001846	0.001722	
9	0.854621	0.622018	0.333649	0.114426	0.001846	0.001722	0.055841	

	y(t-1)	y(t)	y(t+1)
0	0.771084	0.911432	0.854621
1	0.911432	0.854621	0.622018
2	0.854621	0.622018	0.333649
3	0.622018	0.333649	0.114426
4	0.333649	0.114426	0.001846
5	0.114426	0.001846	0.001722
6	0.001846	0.001722	0.055841
7	0.001722	0.055841	0.199390
8	0.055841	0.199390	0.119530
9	0.199390	0.119530	0.261355

```
[125]: train_values = train_val.values

input_window = 0
X_train = train_values[:,input_window:-1]
y_train = train_values[:, -1]
```

```
test_values = test_val.values
X_test = test_values[:,input_window:-1]
y_test = test_values[:, -1]
```

```
[126]: train_dataset = CustomDataset(torch.from_numpy(X_train).float(), torch.
        ↪from_numpy(y_train).float().unsqueeze(1))
test_dataset = CustomDataset(torch.from_numpy(X_test).float(), torch.
        ↪from_numpy(y_test).float().unsqueeze(1))

batch_size = 32
train_loader = DataLoader(dataset=train_dataset, batch_size=batch_size)
test_loader = DataLoader(dataset=test_dataset, batch_size=1)

n_input = X_train.shape[1]
n_output = 1
hidden_neurons = 20

model = NeuralNetwork(n_input=n_input, n_output=n_output, ↪
        ↪hidden_neurons=hidden_neurons, hidden_activation='sigmoid', ↪
        ↪output_activation='softmax')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(device)
lr = 0.05
epochs = 1000

history, y_hat = train(model, train_loader, epochs, device, lr)

y_result = multi_step(model, X_test, cod='binary', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')
```

cuda

```
7%|          | 68/1000 [00:00<00:05, 160.40it/s]
```

Epoch 51/1000, Loss: 0.0390

```
12%|         | 124/1000 [00:00<00:04, 175.89it/s]
```

Epoch 101/1000, Loss: 0.0208
 18%| | 183/1000 [00:01<00:04, 187.20it/s]
 Epoch 151/1000, Loss: 0.0107
 22%| | 221/1000 [00:01<00:04, 179.09it/s]
 Epoch 201/1000, Loss: 0.0088
 28%| | 279/1000 [00:01<00:03, 186.66it/s]
 Epoch 251/1000, Loss: 0.0085
 32%| | 317/1000 [00:01<00:03, 182.25it/s]
 Epoch 301/1000, Loss: 0.0085
 37%| | 371/1000 [00:02<00:03, 158.47it/s]
 Epoch 351/1000, Loss: 0.0084
 42%| | 420/1000 [00:02<00:03, 150.59it/s]
 Epoch 401/1000, Loss: 0.0083
 47%| | 468/1000 [00:02<00:03, 147.29it/s]
 Epoch 451/1000, Loss: 0.0083
 53%| | 533/1000 [00:03<00:03, 154.08it/s]
 Epoch 501/1000, Loss: 0.0083
 56%| | 564/1000 [00:03<00:03, 142.06it/s]
 Epoch 551/1000, Loss: 0.0082
 63%| | 627/1000 [00:03<00:02, 149.40it/s]
 Epoch 601/1000, Loss: 0.0082
 68%| | 676/1000 [00:04<00:02, 152.27it/s]
 Epoch 651/1000, Loss: 0.0082
 72%| | 724/1000 [00:04<00:01, 150.33it/s]
 Epoch 701/1000, Loss: 0.0082
 77%| | 773/1000 [00:04<00:01, 153.21it/s]
 Epoch 751/1000, Loss: 0.0081
 82%| | 824/1000 [00:05<00:01, 159.80it/s]
 Epoch 801/1000, Loss: 0.0081
 88%| | 877/1000 [00:05<00:00, 166.28it/s]
 Epoch 851/1000, Loss: 0.0081
 91%| | 911/1000 [00:05<00:00, 146.35it/s]

Epoch 901/1000, Loss: 0.0081

97%| | 973/1000 [00:06<00:00, 147.13it/s]

Epoch 951/1000, Loss: 0.0081

100%| | 1000/1000 [00:06<00:00, 156.77it/s]

Epoch 1001/1000, Loss: 0.0081

[0. 0. 0. 1.] 0 0

[0. 0. 0. 0.] 1 1

[0. 0. 0. 1.] 2 2

[0. 0. 1. 0.] 3 3

[0. 0. 1. 1.] 4 4

[0. 1. 0. 0.] 5 5

[0. 1. 0. 1.] 6 6

[0. 1. 1. 0.] 7 7

[0. 1. 1. 1.] 8 8

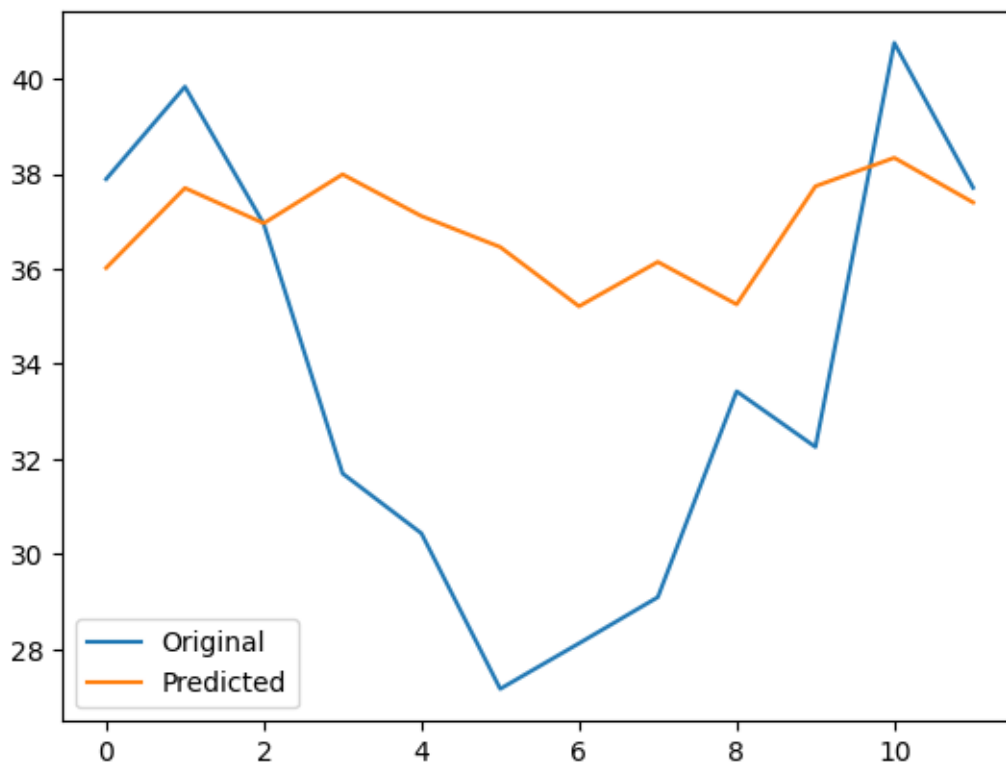
[1. 0. 0. 0.] 9 10

[1. 0. 1. 0.] 10 11

[1. 0. 1. 1.] 11 12

Erro MSE = 26.487

Erro MAE = 4.204

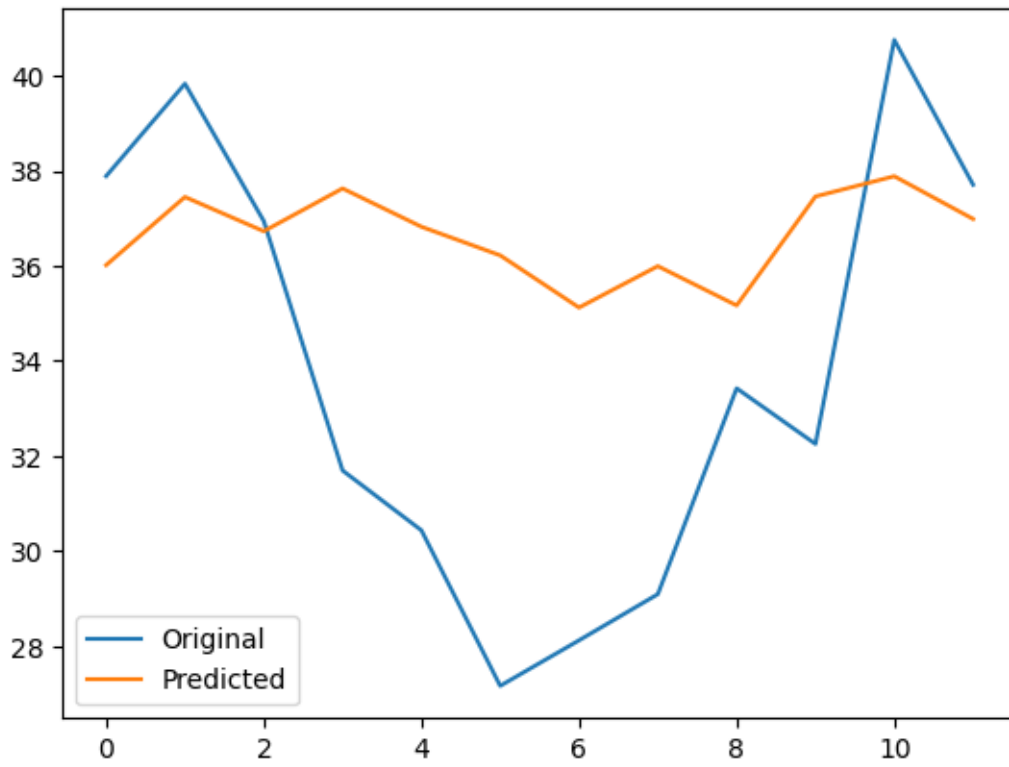


```
[122]: y_result = multi_step(model, X_test, cod='binary', device=device)
orig_y_eval = y_test*(_max-_min) + _min
orig_y_hat = y_result*(_max-_min) + _min
mae_error = mean_absolute_error(orig_y_eval, orig_y_hat)
mse_error = mean_squared_error(orig_y_eval, orig_y_hat)

plt.plot(orig_y_eval)
plt.plot(orig_y_hat)
plt.legend(['Original', 'Predicted'])

print(f'Erro MSE = {round(mse_error,3)} \nErro MAE = {round(mae_error,3)}')
```

```
[0. 0. 0. 1.] 0 0
[0. 0. 0. 0.] 1 1
[0. 0. 0. 1.] 2 2
[0. 0. 1. 0.] 3 3
[0. 0. 1. 1.] 4 4
[0. 1. 0. 0.] 5 5
[0. 1. 0. 1.] 6 6
[0. 1. 1. 0.] 7 7
[0. 1. 1. 1.] 8 8
[1. 0. 0. 0.] 9 10
[1. 0. 1. 0.] 10 11
[1. 0. 1. 1.] 11 12
Erro MSE = 25.22
Erro MAE = 4.19
```



[]: